

CST-407 Activity 6 Implementing REST API Security with Java Web Tokens (JWT)

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

08/25/2025

Table of Contents

Introduction.....	4
Step 1 — Create & seed the database	5
1.1 – Database Created	5
1.2 – Users Table Data	6
1.3 – Orders Table Data	7
1.4 – DB Designer / ER view	8
Step 2 — Run the Starter Application (No Security)	9
2.1 – Application Properties.....	9
2.2 – Application Running	10
2.3 – Orders Without Security.....	11
Step 3 — Run the JavaScript client (no security yet).....	12
3.1 – Client Project Opened	12
3.2 – Client Base URL	13
3.3 – API CORS Enabled.....	14
3.4 – Live Server Running.....	15
3.5 – Orders loaded in client (no security).....	16
Step 4 — Add JWT Dependencies and Security Configuration	17
4.1 – JWT and Security Dependencies	17
4.2 – Create JwtTokenProvider	18
4.2.1 – Configure JWT Properties.....	19
4.3 – Create the JWT authentication filter	20
4.4 – Create Spring Security config	21
4.5 – Create Custom User Details Service.....	22
4.6 – Create Custom User Details.....	23
4.7 – UsersController (JWT login + BCrypt register)	24
4.8 –Register user in Postman	25
4.9 –Login in Postman to get the JWT	26
4.10 –Call protected endpoint with Bearer token	27
4.11 –Client login to obtain token.....	28
4.12 –Client gets orders with JWT.....	29
4.13 –Create order from client with JWT	30
Step 5 – Secure Orders API with Authenticated User Context.....	31

5.2 – Orders filtered by authenticated user.....	32
5.2.1 – Retrieve Orders by Customer Id (Protected)	33
5.3 – Negative test: protected endpoint without token	34
Step 5.4 – Negative Test (Invalid Token)	35
5.5 – Verify authorization from the client (not logged in)	36
5.6 – Verify authorization from the client (logged in)	37
Step 6 – Add Role-Based Authorization.....	38
6.1 – Add a <code>role</code> field to <code>UserModel</code> (default: <code>USER</code>)	38
6.2 – Put the user's <code>role</code> in the JWT and load it into Spring Security	39
6.3 – Restrict write operations to <code>ADMIN</code>	40
6.4 Create an Admin User	41
6.5 – <code>USER</code> blocked from creating order.....	42
6.6 – Verify <code>ADMIN</code> <i>can</i> create an order	43
6.7 – Update Order (PUT) with Auth	44
6.8 – Delete Order (DELETE) with Auth.....	45
Summary of Key Concepts.....	46

Introduction

The purpose of Activity 6 is to extend the Orders application by adding **role-based access control (RBAC)** using Spring Security and JSON Web Tokens (JWT). In earlier activities, authentication is implemented so users can log in and receive a token. This activity builds on that by assigning each user a specific role, such as `USER` or `ADMIN`, and enforcing permissions in the application's endpoints. Through this process, I learn how to include role information in the JWT, configure the security filter chain to restrict access, and test how different roles interact with the system. By the end of the activity, the application is able to distinguish between normal users and administrators, ensuring that only admins can perform sensitive actions like creating or deleting orders, while regular users are limited to basic read operations.

Step 1 — Create & seed the database

1.1 – Database Created

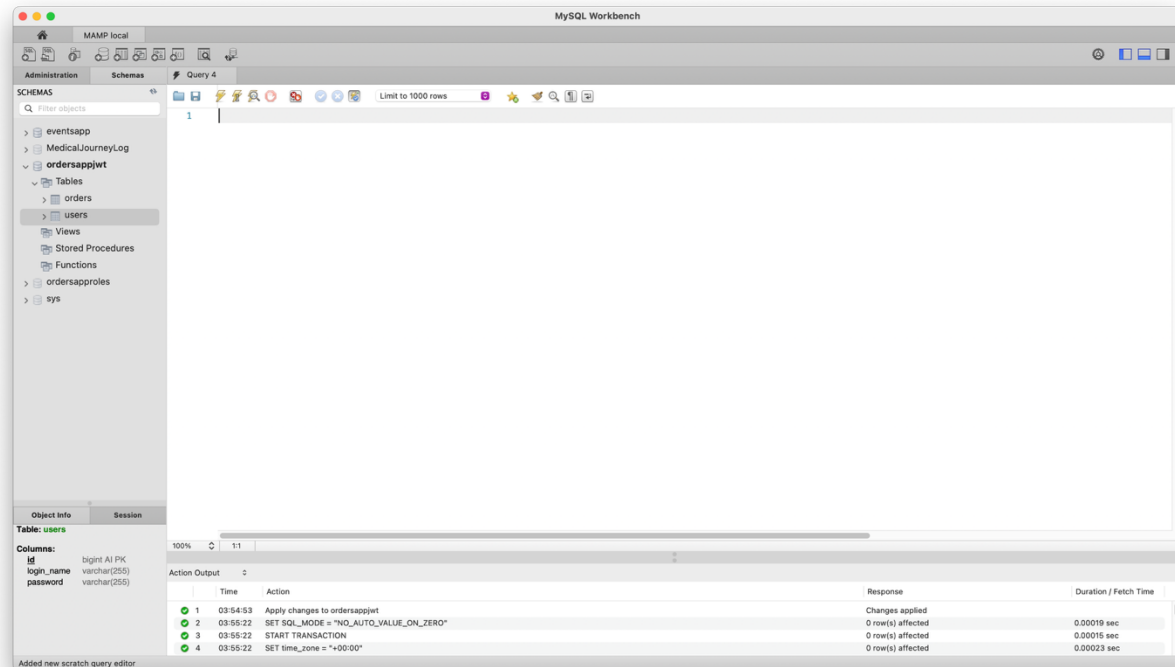


Figure 1 The ordersappjwt database created successfully in MySQL Workbench.

1.2 – Users Table Data

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema hierarchy, including the 'users' table under the 'ordersappjvt' database. The main window shows the 'users' table with the following data:

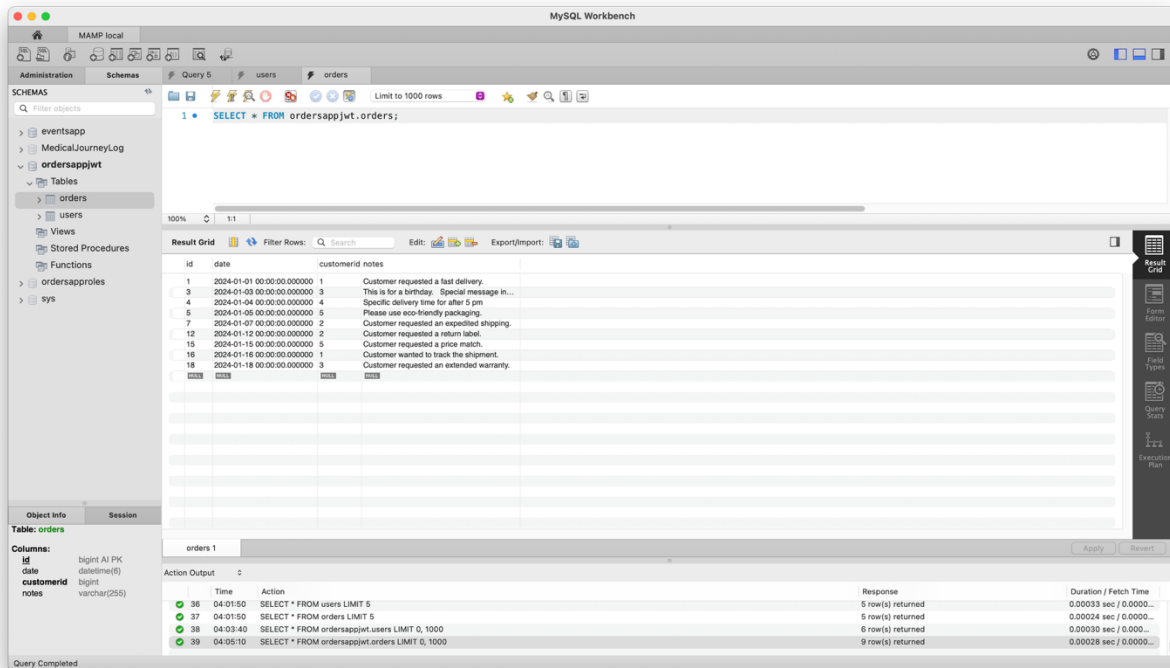
id	login_name	password
1	alice	password123
2	bob	!@h\$0berichandflamou\$
3	charlie	new_password
4	david	%!tabc
5	eva	overveerveve
10	shad	\$2u\$10\$8m8dy0h/KrYp\$uKy0vOcxq\$XibCK...

The bottom panel shows the 'Action Output' tab with the following log:

Action	Time	Response	Duration / Fetch Time
SHOW TABLES	04:01:50	2 row(s) returned	0.00095 sec / 0.000...
SELECT * FROM users LIMIT 5	04:01:50	5 row(s) returned	0.00033 sec / 0.0000...
SELECT * FROM orders LIMIT 5	04:01:50	5 row(s) returned	0.00024 sec / 0.0000...
SELECT * FROM ordersappjvt.users LIMIT 0, 1000	04:03:40	6 row(s) returned	0.00030 sec / 0.000...

Figure 2 The users table populated with seed data, showing accounts such as Alice, Bob, and Charlie.

1.3 – Orders Table Data



The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including tables like `eventsapp`, `MedicalJourneyLog`, `ordersappjwv`, `orders`, `users`, `Views`, `Stored Procedures`, `Functions`, `ordersapproles`, and `sys`. The `orders` table is selected. The main area shows the SQL query: `SELECT * FROM ordersappjwv.orders;`. The `Result Grid` displays the following data:

id	date	customerid	notes
1	2024-01-01 00:00:00.000000	1	Customer requested a fast delivery.
3	2024-01-03 00:00:00.000000	3	This is for a birthday. Special message in...
4	2024-01-04 00:00:00.000000	4	Specific delivery time for after 5 pm
5	2024-01-05 00:00:00.000000	5	Please use eco-friendly packaging.
7	2024-01-07 00:00:00.000000	2	Customer requested an expedited shipping.
12	2024-01-12 00:00:00.000000	2	Customer requested a return label.
15	2024-01-15 00:00:00.000000	5	Customer requested a price match.
16	2024-01-16 00:00:00.000000	1	Customer wanted to track the shipment.
18	2024-01-18 00:00:00.000000	3	Customer requested an extended warranty.

The bottom panel shows the `Table: orders` structure with columns: `id` (bigint AI PK), `date` (datetime(6)), `customerid` (bigint), and `notes` (varchar(255)). The `Action Output` panel shows the execution of the query, indicating that 9 rows were returned.

Figure 3 The orders table seeded with initial customer orders linked to the users table by foreign key.

1.4 – DB Designer / ER view

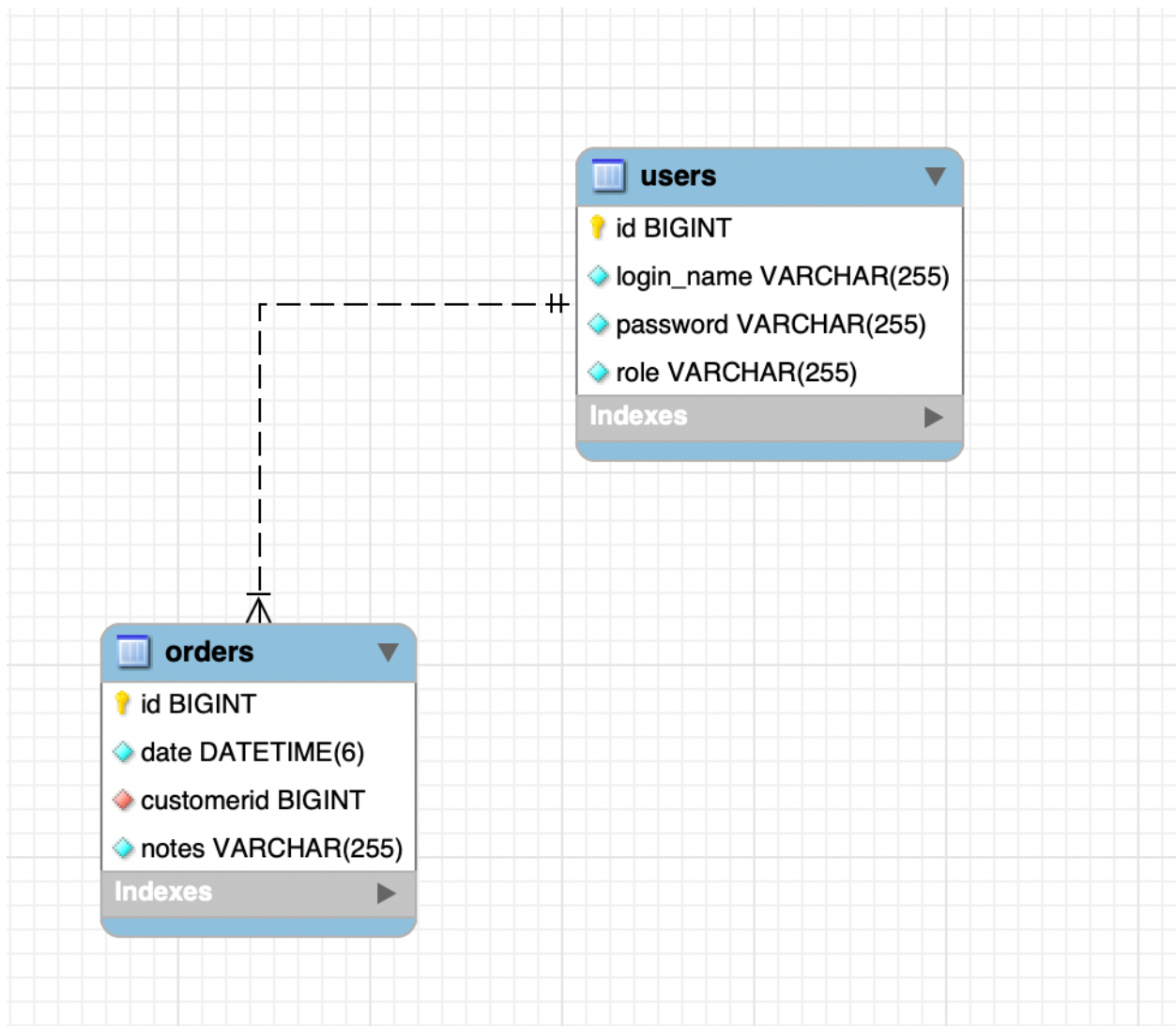


Figure 4 Designer view showing foreign key `orders.customerid` referencing `users.id`.

Step 2 — Run the Starter Application (No Security)

2.1 – Application Properties

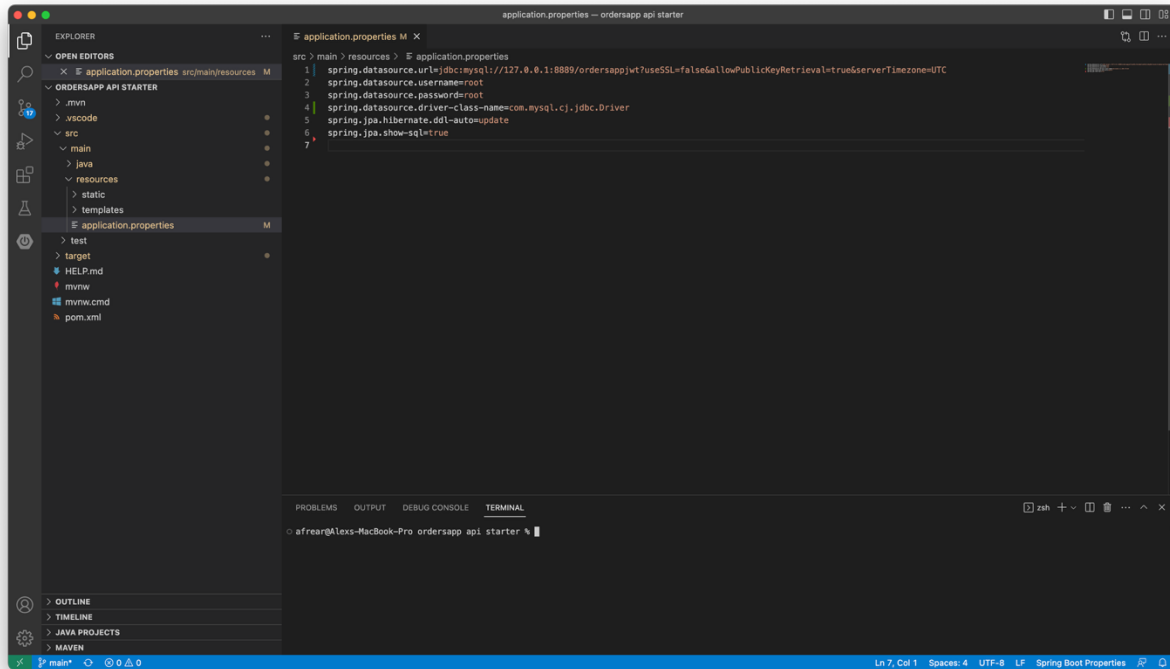


Figure 5 The Spring Boot application configured to connect to the ordersappjwt database using MySQL.

2.2 – Application Running

The screenshot shows an IDE with the Explorer view on the left, displaying the project structure of 'ordersapp api starter'. The 'application.properties' file is open in the Editor view, showing the following configuration:

```
src > main > resources > application.properties
1 spring.datasource.url=jdbc:mysql://127.0.0.1:8889/ordersappjw?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
2 spring.datasource.username=root
3 spring.datasource.password=root
4 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
5 spring.jpa.hibernate.ddl-auto=update
6 spring.jpa.show-sql=true
7
```

The Terminal view at the bottom shows the output of the application startup. The output is as follows:

```
2025-08-23T05:04:50.268-07:00 INFO 76700 [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : DevTools property defaults active! Set 'spring.devtools.add-properties' to 'false' to disable
2025-08-23T05:04:50.268-07:00 INFO 76700 [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
2025-08-23T05:04:50.717-07:00 INFO 76700 [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2025-08-23T05:04:50.764-07:00 INFO 76700 [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 42 ms. Found 2 JPA repository interfaces.
2025-08-23T05:04:51.165-07:00 INFO 76700 [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8888 (http)
2025-08-23T05:04:51.175-07:00 INFO 76700 [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-08-23T05:04:51.175-07:00 INFO 76700 [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.24]
2025-08-23T05:04:51.209-07:00 INFO 76700 [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-08-23T05:04:51.218-07:00 INFO 76700 [ restartedMain] o.s.c.s.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 948 ms
2025-08-23T05:04:51.324-07:00 INFO 76700 [ restartedMain] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: default]
2025-08-23T05:04:51.365-07:00 INFO 76700 [ restartedMain] o.hibernate.Version : HHH000412: Hibernate ORM core version 6.5.2.Final
2025-08-23T05:04:51.391-07:00 INFO 76700 [ restartedMain] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-level cache disabled
2025-08-23T05:04:51.607-07:00 INFO 76700 [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2025-08-23T05:04:51.620-07:00 INFO 76700 [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2025-08-23T05:04:51.768-07:00 INFO 76700 [ restartedMain] com.zaxxer.hikari.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@2a6306
2025-08-23T05:04:51.769-07:00 INFO 76700 [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2025-08-23T05:04:52.373-07:00 INFO 76700 [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2025-08-23T05:04:52.485-07:00 INFO 76700 [ restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2025-08-23T05:04:52.662-07:00 WARN 76700 [ restartedMain] jakarta.persistence.spi.PersistenceUnitProviderConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2025-08-23T05:04:52.888-07:00 INFO 76700 [ restartedMain] o.s.b.o.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2025-08-23T05:04:52.908-07:00 INFO 76700 [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8888 (http) with context path '/'
2025-08-23T05:04:52.915-07:00 INFO 76700 [ restartedMain] c.s.ordersapp.OrdersappApplication : Started OrdersappApplication in 2.958 seconds (process running for 3.35 s)
```

Figure 6 The starter OrdersApp Spring Boot application running on localhost:8080 with no security enabled.

2.3 – Orders Without Security

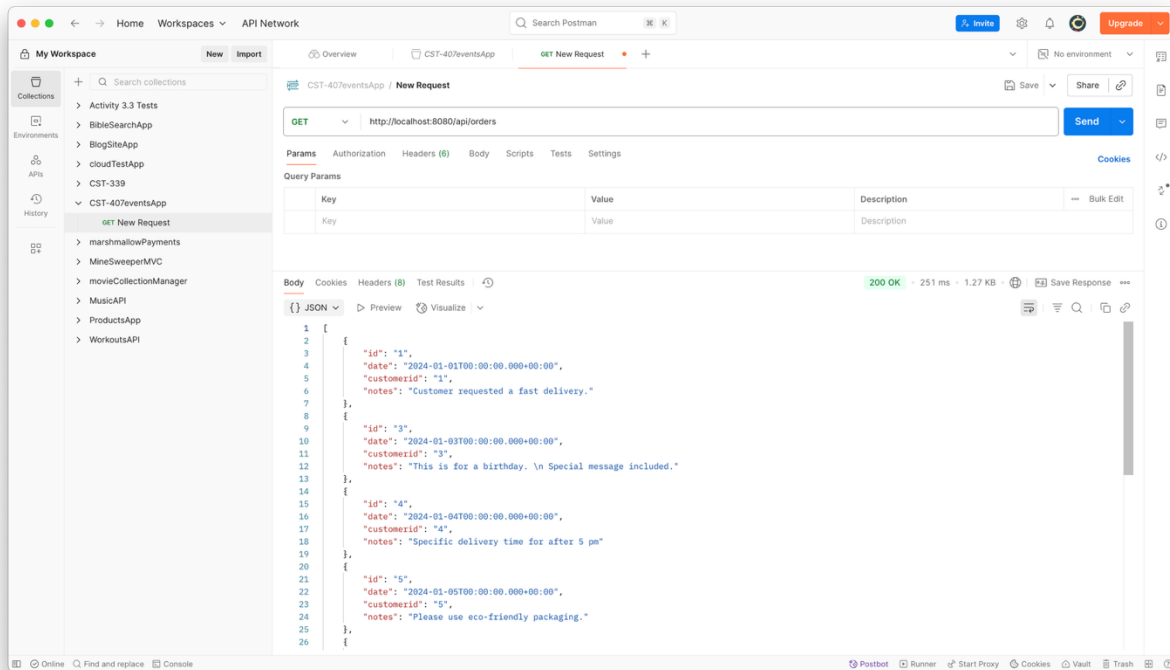


Figure 7 The `/api/orders` endpoint returns all orders successfully without authentication in the starter app.

Step 3 — Run the JavaScript client (no security yet)

3.1 – Client Project Opened

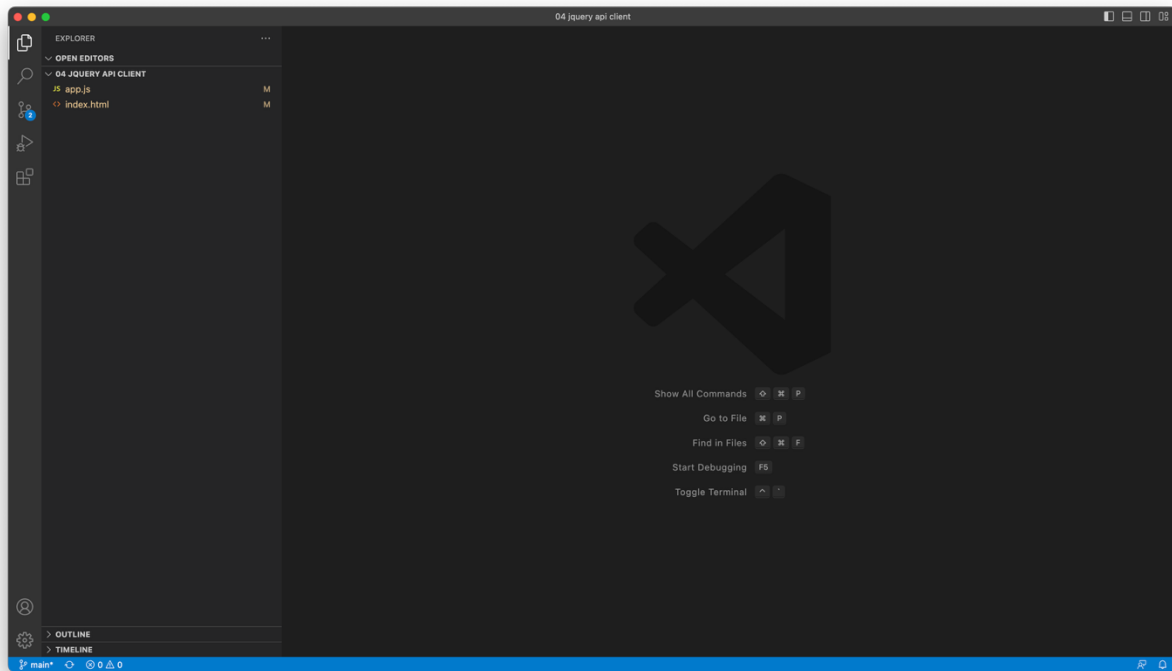
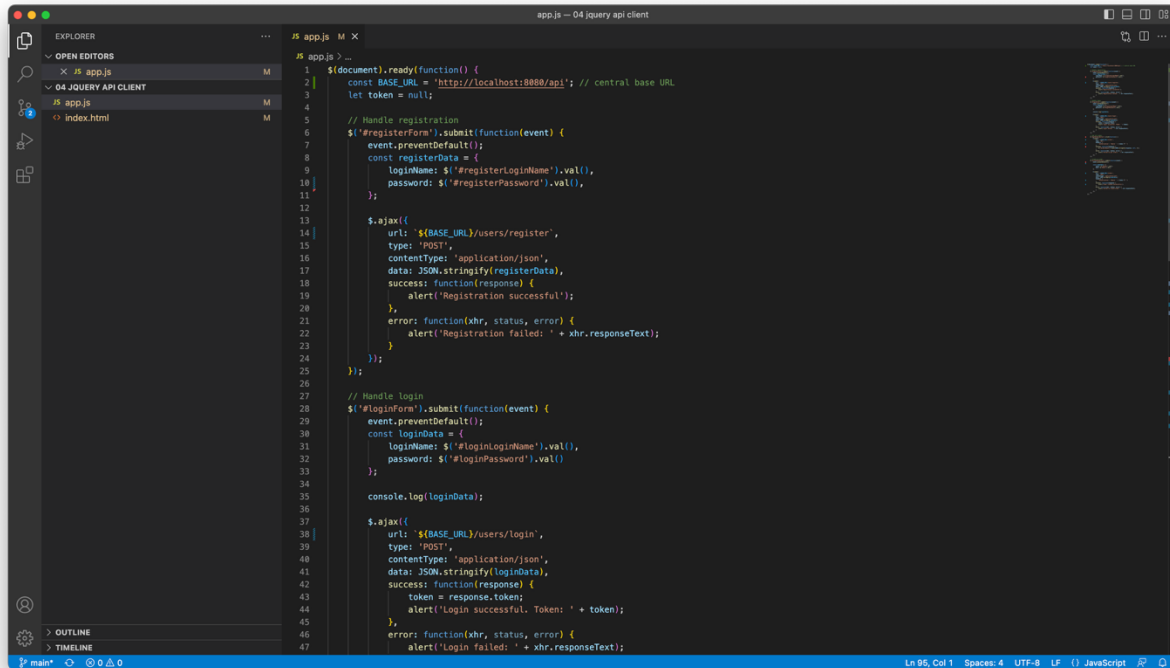


Figure 8 The jQuery client project opened in VS Code showing `index.html` and `app.js`.

3.2 – Client Base URL



```
1 $(document).ready(function() {
2   const BASE_URL = 'http://localhost:8080/api'; // central base URL
3   let token = null;
4
5   // Handle registration
6   $('#registerForm').submit(function(event) {
7     event.preventDefault();
8     const registerData = {
9       loginName: $('#registerLoginName').val(),
10      password: $('#registerPassword').val(),
11    };
12
13    $.ajax({
14      url: `${BASE_URL}/users/register`,
15      type: 'POST',
16      contentType: 'application/json',
17      data: JSON.stringify(registerData),
18      success: function(response) {
19        alert('Registration successful');
20      },
21      error: function(xhr, status, error) {
22        alert('Registration failed: ' + xhr.responseText);
23      }
24    });
25  });
26
27  // Handle login
28  $('#loginForm').submit(function(event) {
29    event.preventDefault();
30    const loginData = {
31      loginName: $('#loginName').val(),
32      password: $('#loginPassword').val()
33    };
34
35    console.log(loginData);
36
37    $.ajax({
38      url: `${BASE_URL}/users/login`,
39      type: 'POST',
40      contentType: 'application/json',
41      data: JSON.stringify(loginData),
42      success: function(response) {
43        token = response.token;
44        alert('Login successful. Token: ' + token);
45      },
46      error: function(xhr, status, error) {
47        alert('Login failed: ' + xhr.responseText);
48      }
49    });
50  });
51 });
```

Figure 9 `app.js` configured to call the Orders API via a single `BASE_URL` of <http://localhost:8080/api>.

3.3 – API CORS Enabled

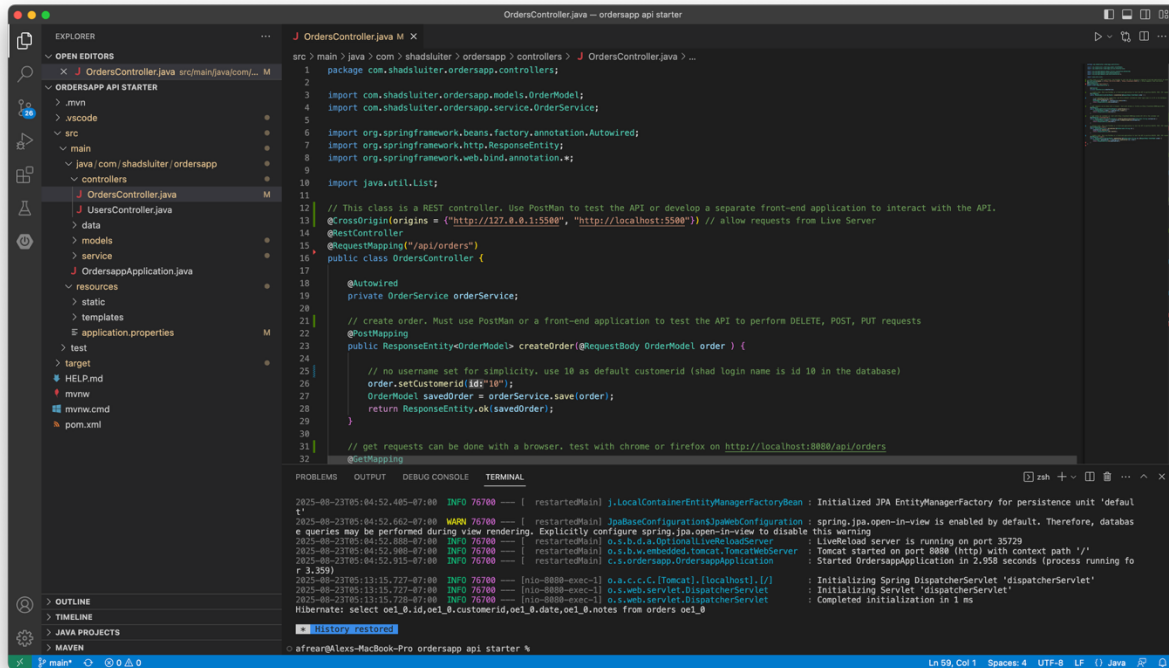


Figure 10 CORS added to the Orders API controller to allow requests from the client running on port 5500.

3.4 – Live Server Running

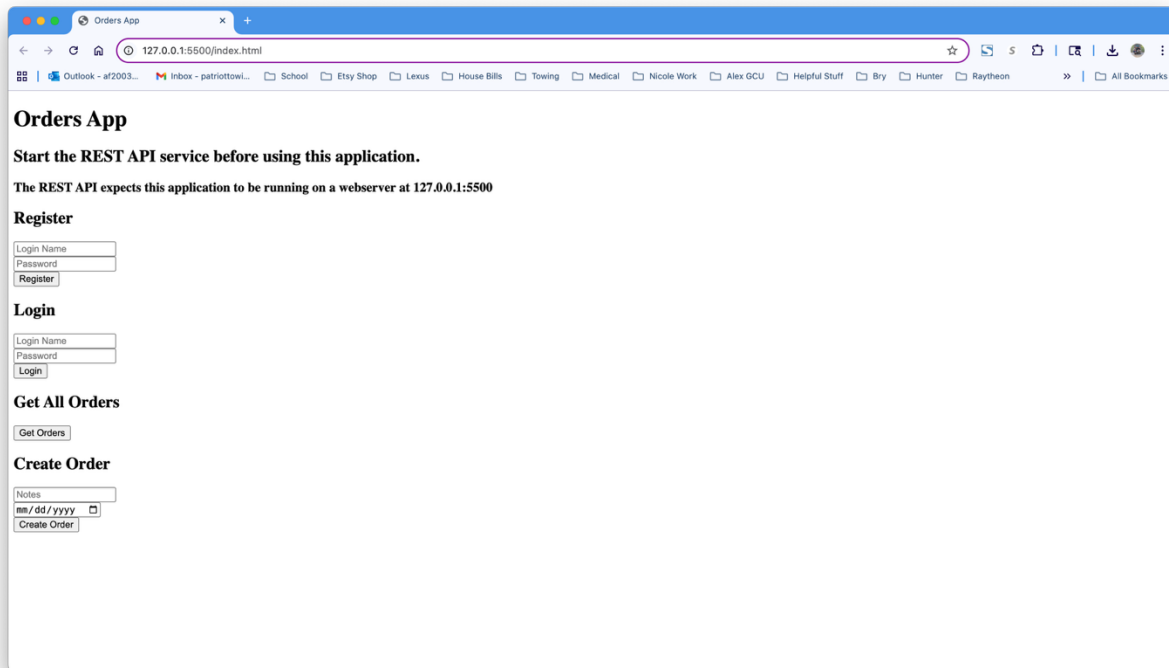


Figure 11 The jQuery client running at `http://127.0.0.1:5500` via Live Server.

3.5 – Orders loaded in client (no security)

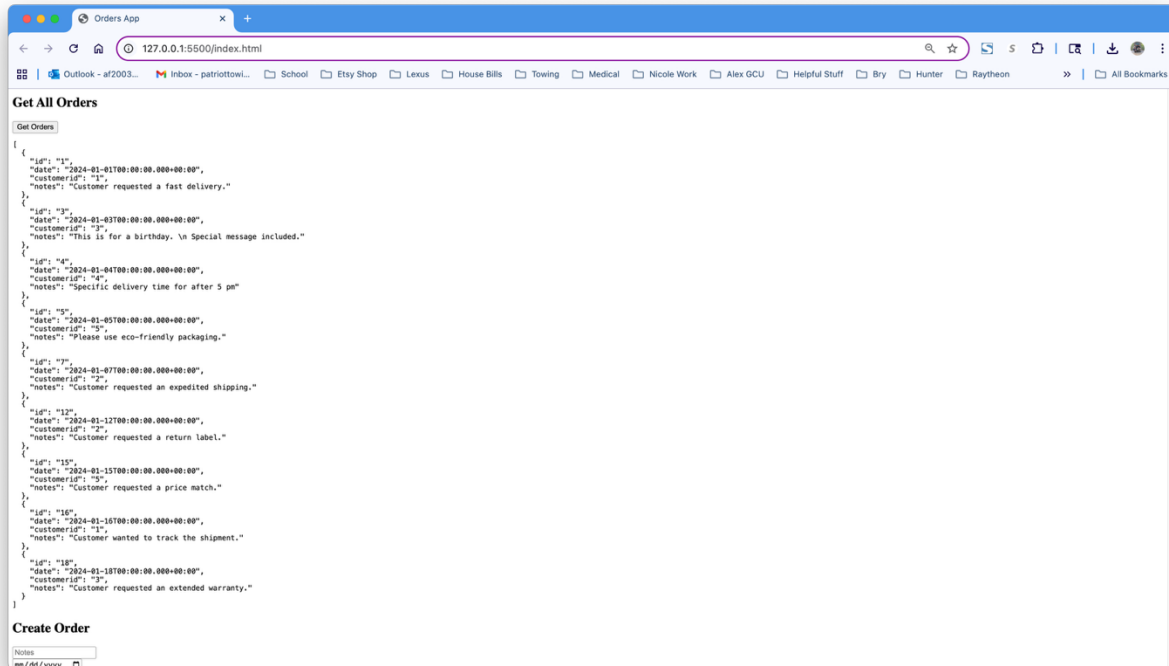


Figure 12 The client retrieves and displays orders from `/api/orders` without authentication.

Step 4 — Add JWT Dependencies and Security Configuration

4.1 – JWT and Security Dependencies

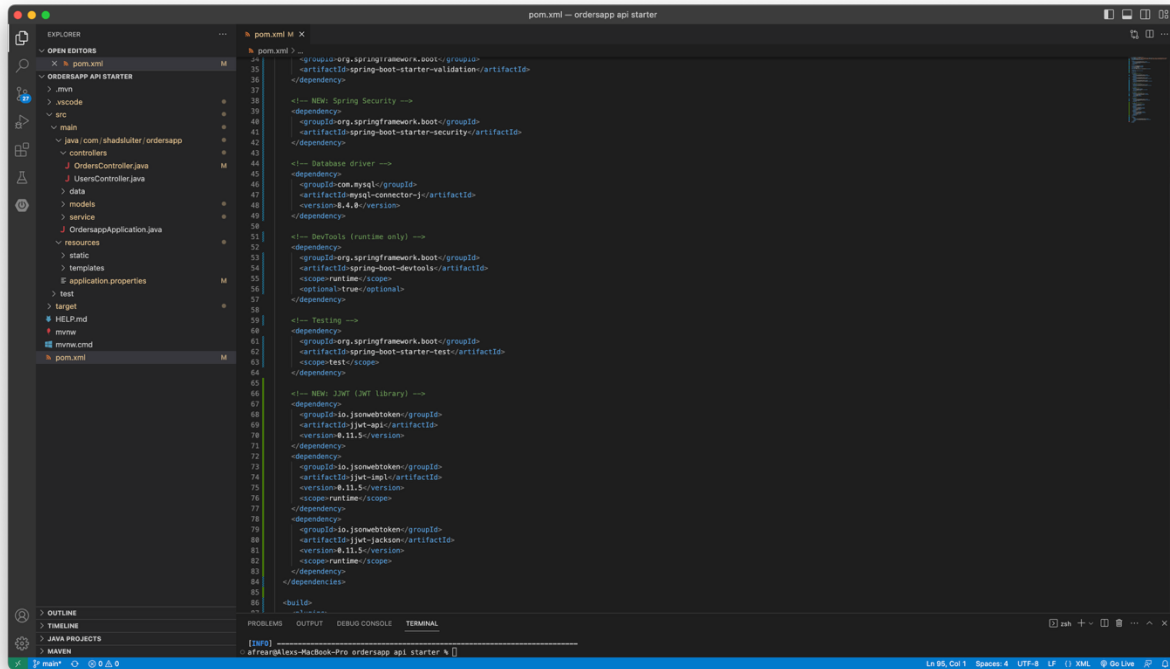


Figure 13 Added Spring Security and JWT dependencies to enable token-based authentication.

4.2 — Create `JwtTokenProvider`

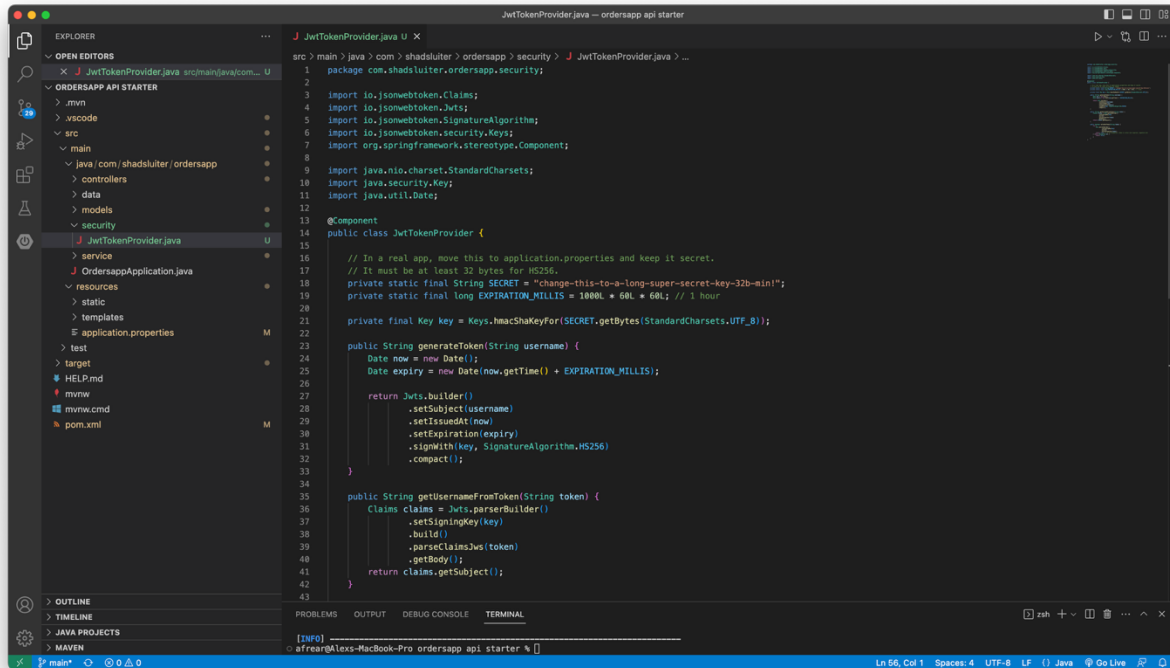
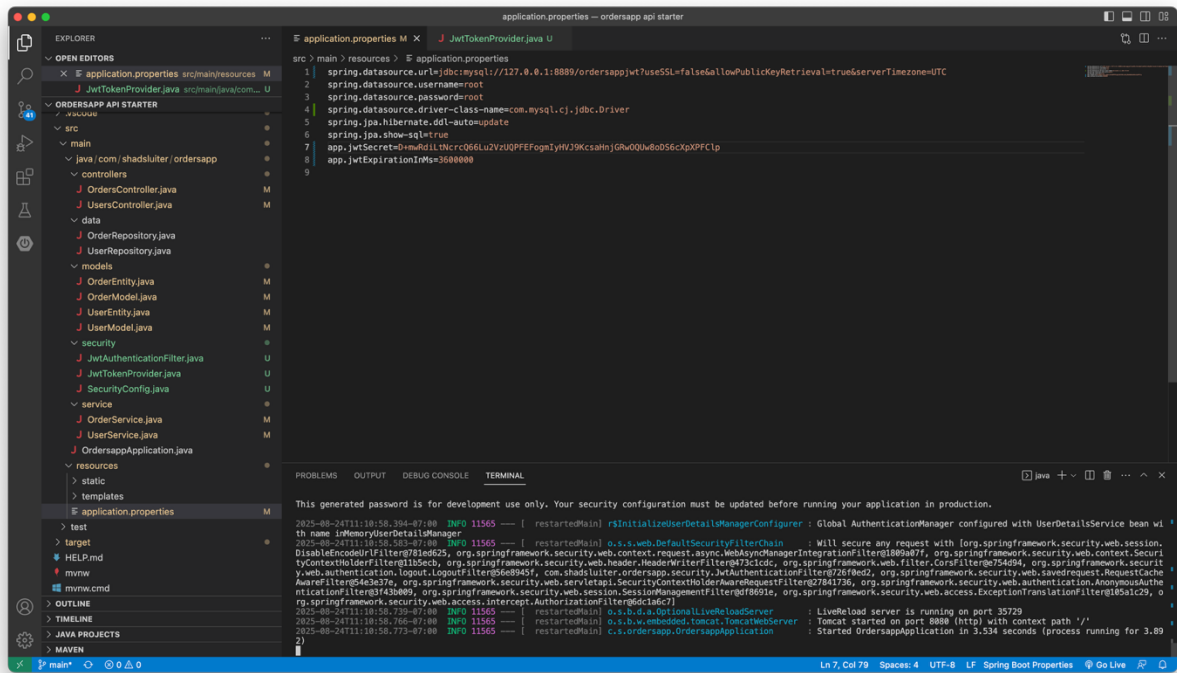


Figure 14 Created `JwtTokenProvider` to generate, parse, and validate JWTs (HS256, 1-hour expiry).

4.2.1 – Configure JWT Properties



The screenshot shows an IDE with the `application.properties` file open in the editor. The file contains the following properties:

```
src > main > resources > application.properties
1 spring.datasource.url=jdbc:mysql://127.0.0.1:8889/ordersappjwt?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
2 spring.datasource.username=root
3 spring.datasource.password=root
4 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
5 spring.jpa.hibernate.ddl-auto=update
6 spring.jpa.show-sql=true
7 app.jwtSecret=0hmdLL1NccrD66Lu2YzUQPFEFogIyHf39KcsahNjGhw00Uw0D56cxpXpFClp
8 app.jwtExpirationInMs=3600000
9
```

The terminal output shows the application starting successfully. The output includes the following lines:

```
2025-08-24T11:10:50.304-07:00 INFO 11565 --- [ restartedMain] r$InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsServiceImpl bean with name i0femrjUserDetailsService
2025-08-24T11:10:50.583-07:00 INFO 11565 --- [ restartedMain] o.s.w.s.DefaultSecurityFilterChain : Will secure any request with [org.springframework.security.web.session.DisableEncodeFilter@7816825, org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFilter@9a877f, org.springframework.security.web.context.SecurityContextHolderFilter@1b5ecb, org.springframework.security.web.header.HeaderWriterFilter@973c0dc, org.springframework.security.web.filter.CorsFilter@754d94, org.springframework.security.web.authentication.logout.LogoutFilter@56e8945f, com.shadsluiter.ordersapp.security.JwtAuthenticationFilter@726f8ed2, org.springframework.security.web.savedrequest.RequestCacheAwareFilter@54eb376, org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFilter@27843736, org.springframework.security.web.authentication.AnonymousAuthenticationFilter@3f43b009, org.springframework.security.web.session.SessionManagementFilter@df8691e, org.springframework.security.web.access.ExceptionTranslationFilter@5a1c29, org.springframework.security.web.access.intercept.AuthorizationFilter@6dc1a6c7]
2025-08-24T11:10:50.729-07:00 INFO 11565 --- [ restartedMain] o.s.b.w.e.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2025-08-24T11:10:50.766-07:00 INFO 11565 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-08-24T11:10:50.773-07:00 INFO 11565 --- [ restartedMain] c.s.ordersapp.OrdersappApplication : Started OrdersappApplication in 3.534 seconds (process running for 3.89s)
```

Figure 15 `application.properties` includes `app.jwtSecret` and `app.jwtExpirationInMs`; `JwtTokenProvider` reads these values to sign and expire tokens.

4.3 — Create the JWT authentication filter

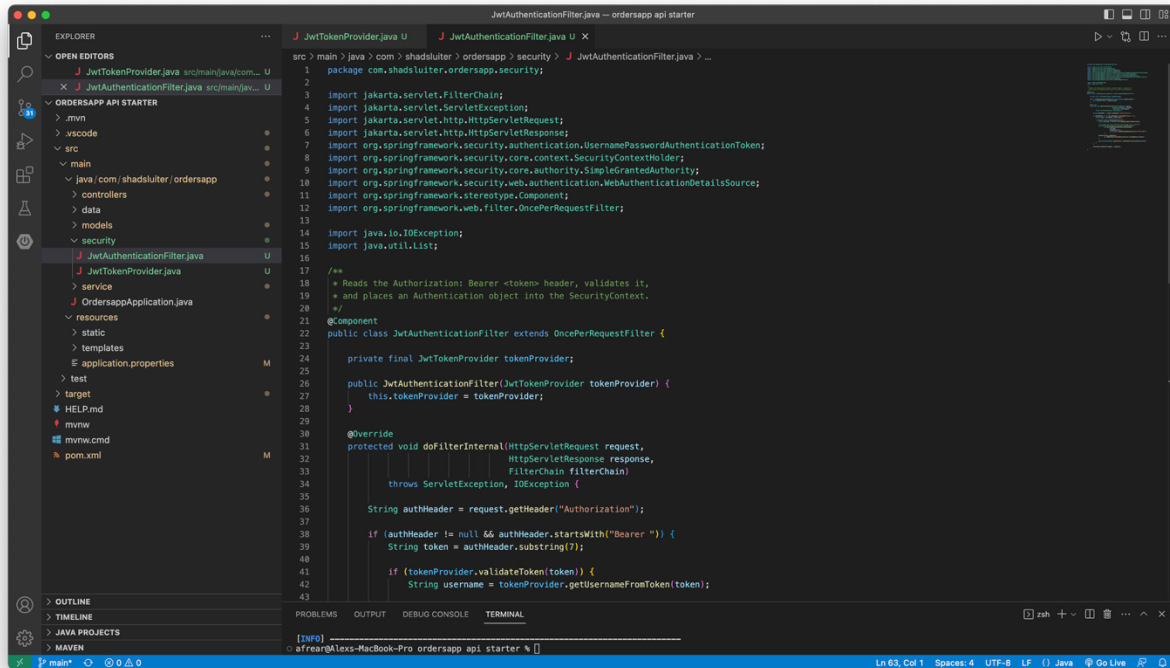


Figure 16 Added `Jwtauthenticationfilter` to read `Authorization: Bearer <token>` and set the authenticated user in the security context.

4.4 — Create Spring Security config

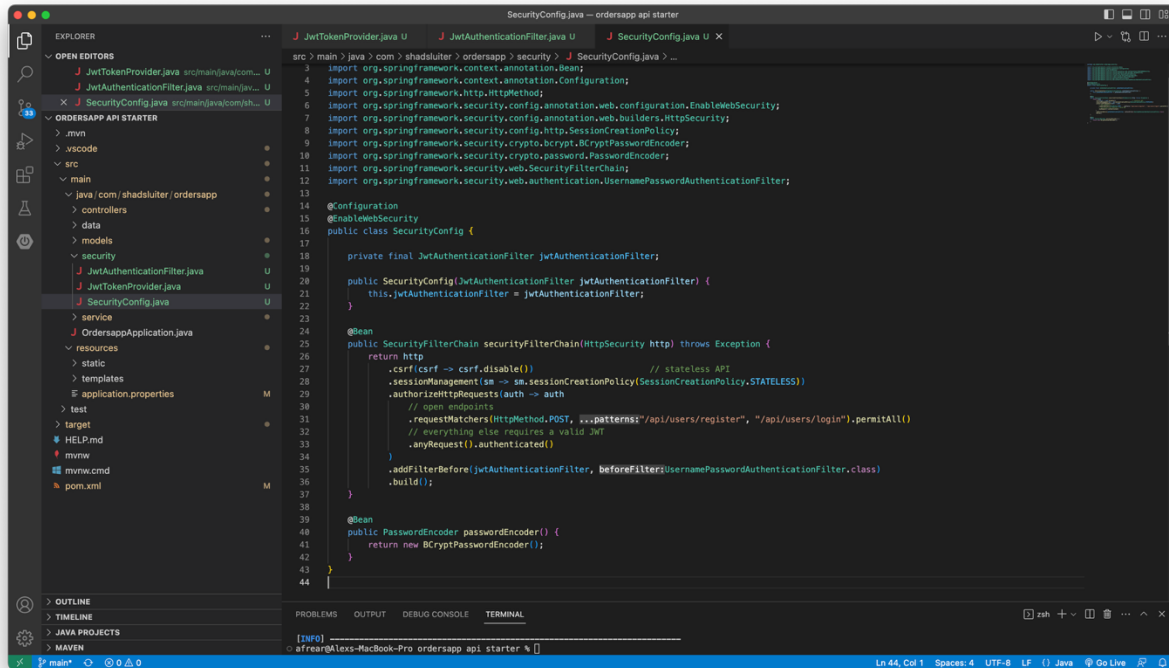


Figure 17 Added `SecurityConfig` to enable stateless JWT security, permit `/api/users/register` and `/api/users/login`, and protect all other endpoints.

4.5 — Create Custom User Details Service

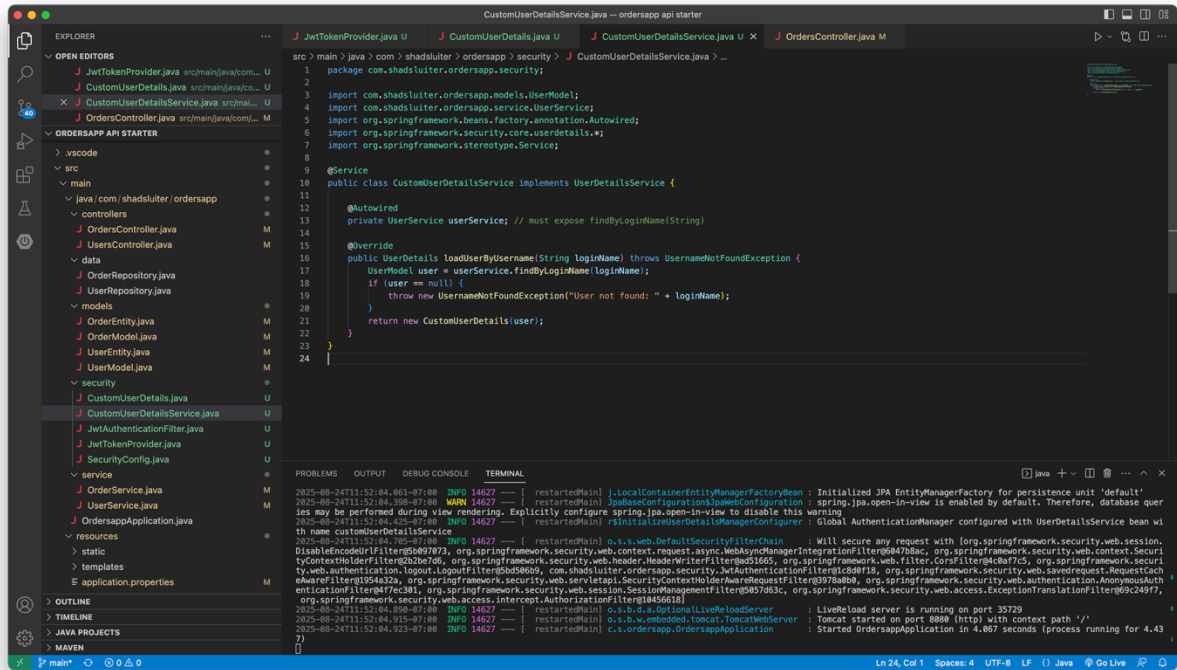


Figure 18 CustomUserDetailsService implements UserDetailsService and overrides loadUserByUsername to load users by loginName from the database.

4.6 — Create Custom User Details

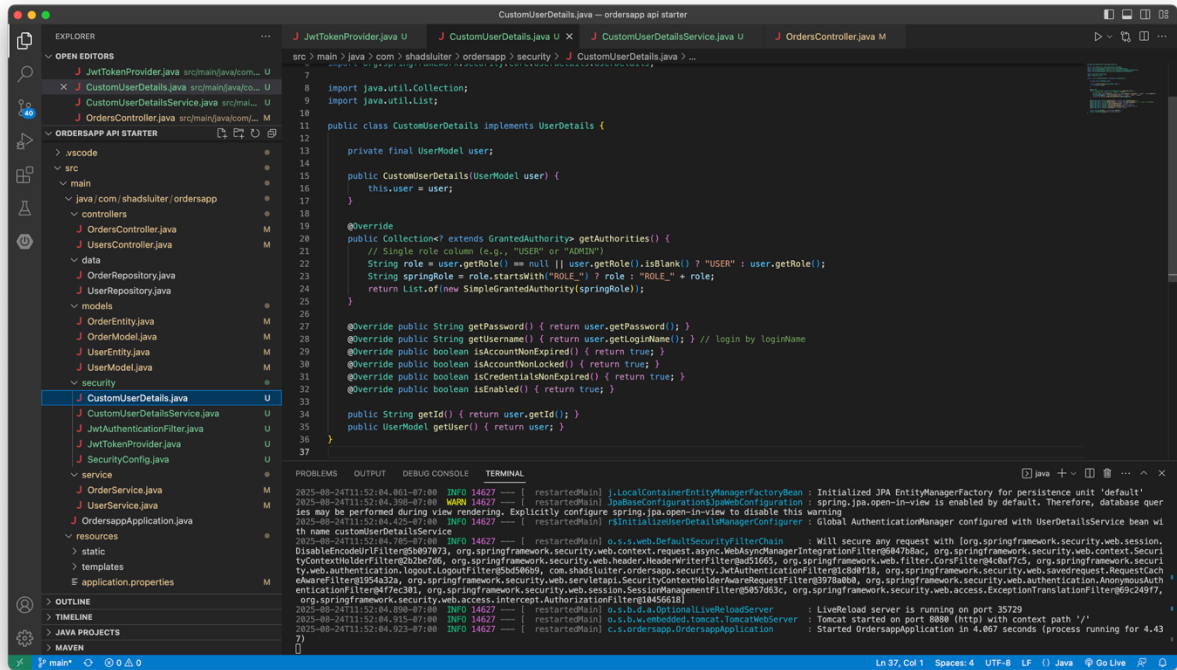


Figure 19 `CustomUserDetails` implements `UserDetails`, exposing `loginName`, `password`, and `ROLE_*` authorities for Spring Security.

4.7 – UsersController (JWT login + BCrypt register)

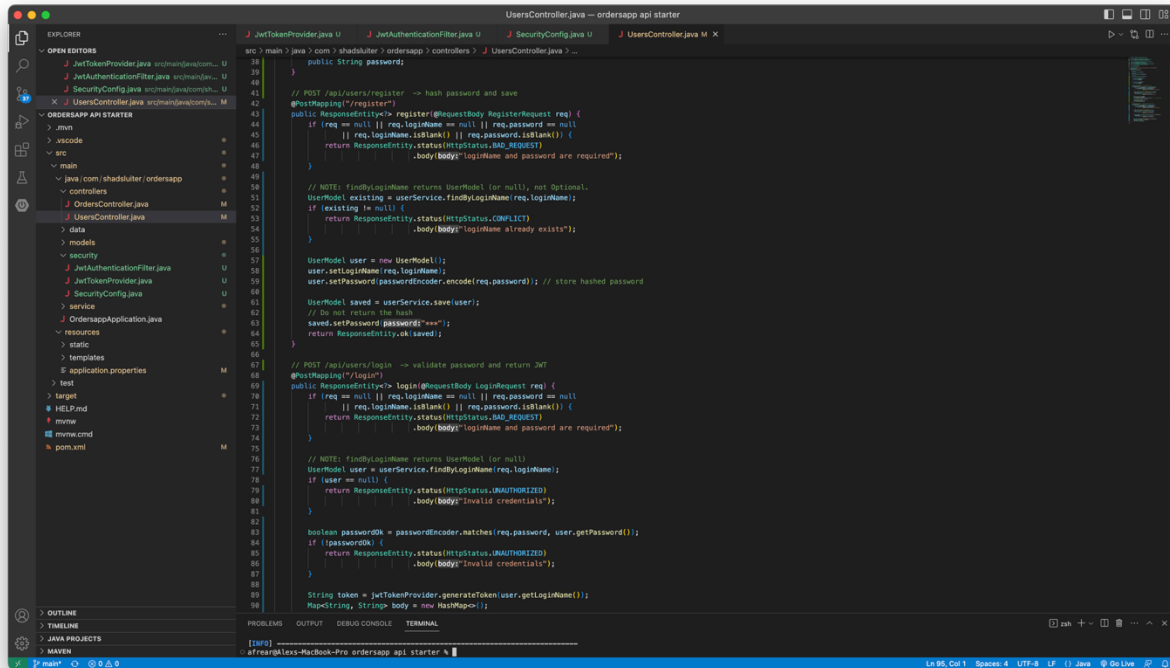


Figure 20 Updated UsersController to hash passwords on register and return a JWT on login (null-safe service lookups).

4.8 —Register user in Postman

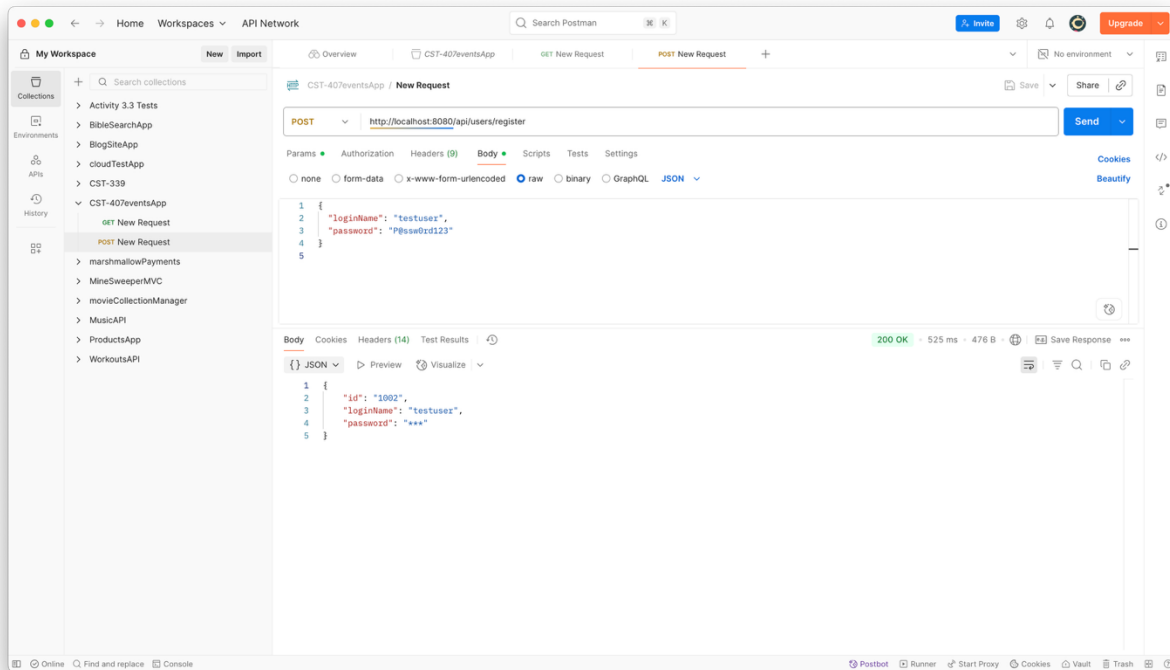


Figure 21 Successful registration: user saved with a BCrypt-hashed password in the database.

4.9 —Login in Postman to get the JWT

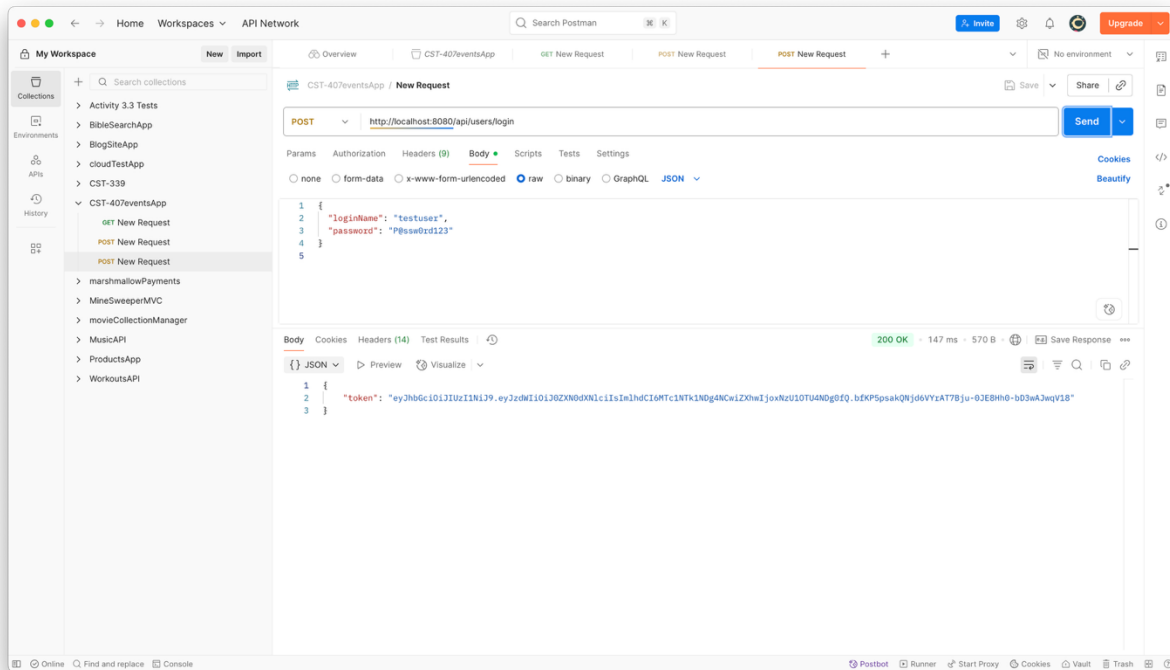


Figure 22 Successful login returning a JWT token to use in the Authorization header.

4.10 —Call protected endpoint with Bearer token

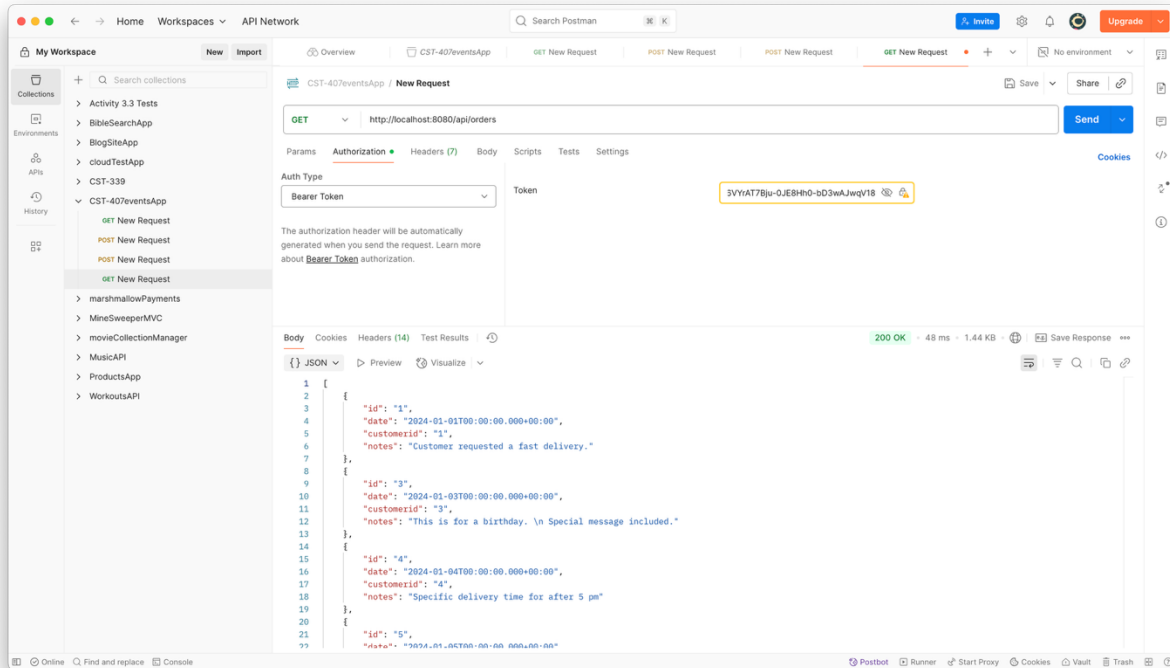


Figure 23 /api/orders accessed successfully using Authorization: Bearer <token>.

4.11 —Client login to obtain token

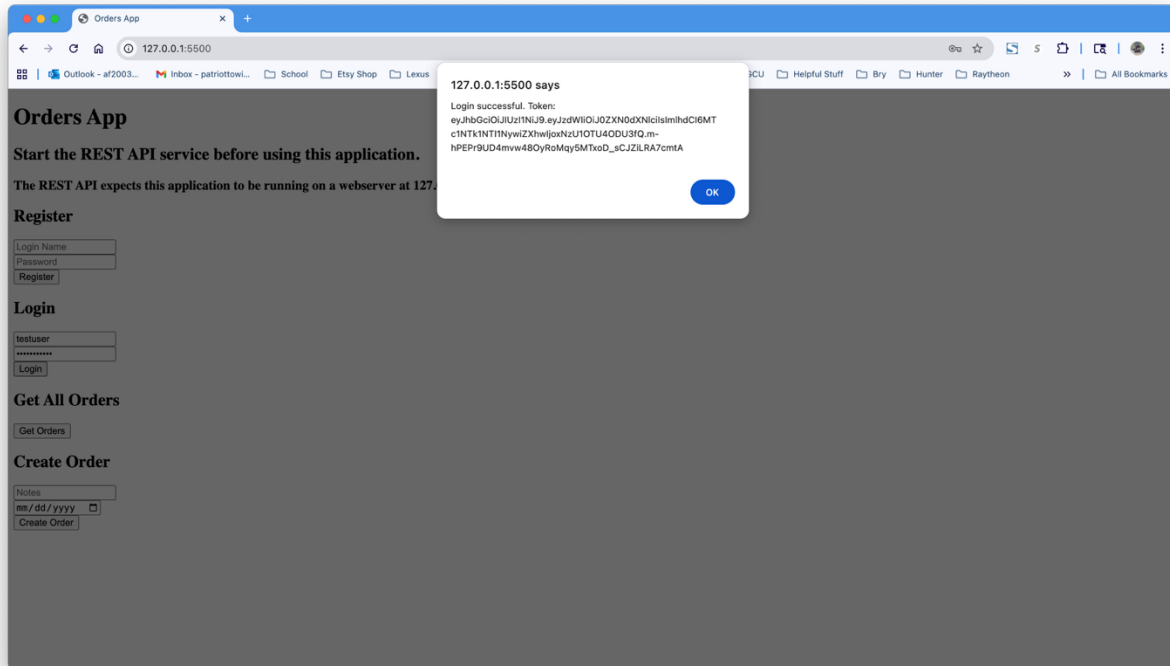


Figure 24 Client login succeeded and received a JWT token from `/api/users/login`.

4.12 —Client gets orders with JWT

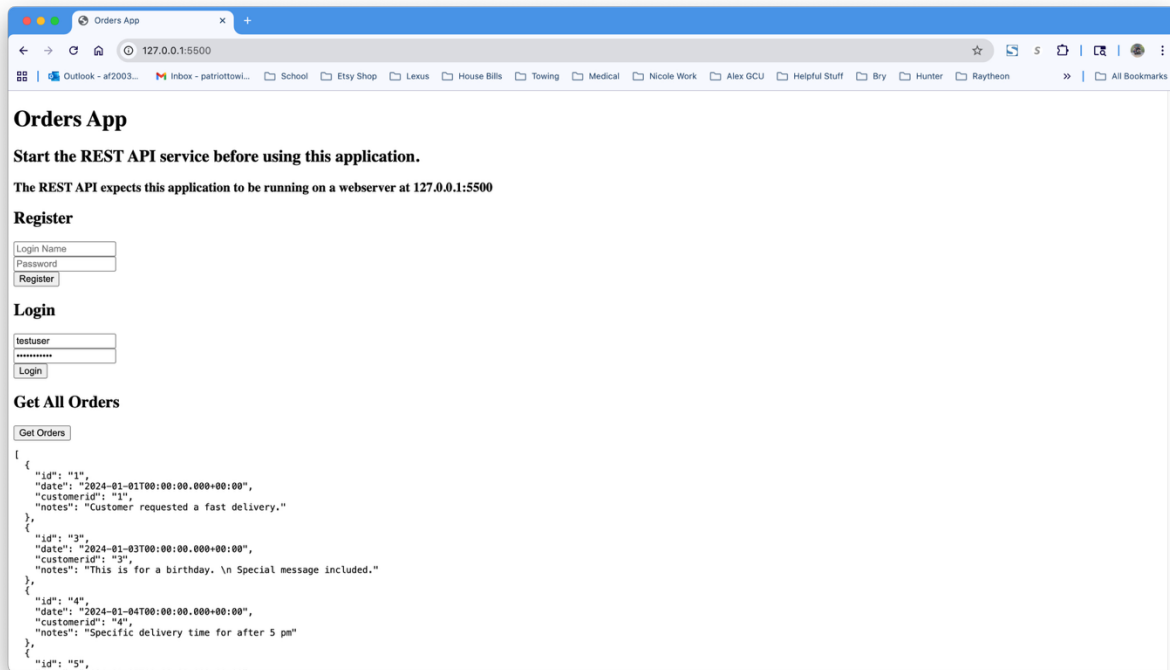


Figure 25 Client successfully calls `/api/orders` using the stored JWT (`Authorization: Bearer <token>`).

4.13 —Create order from client with JWT

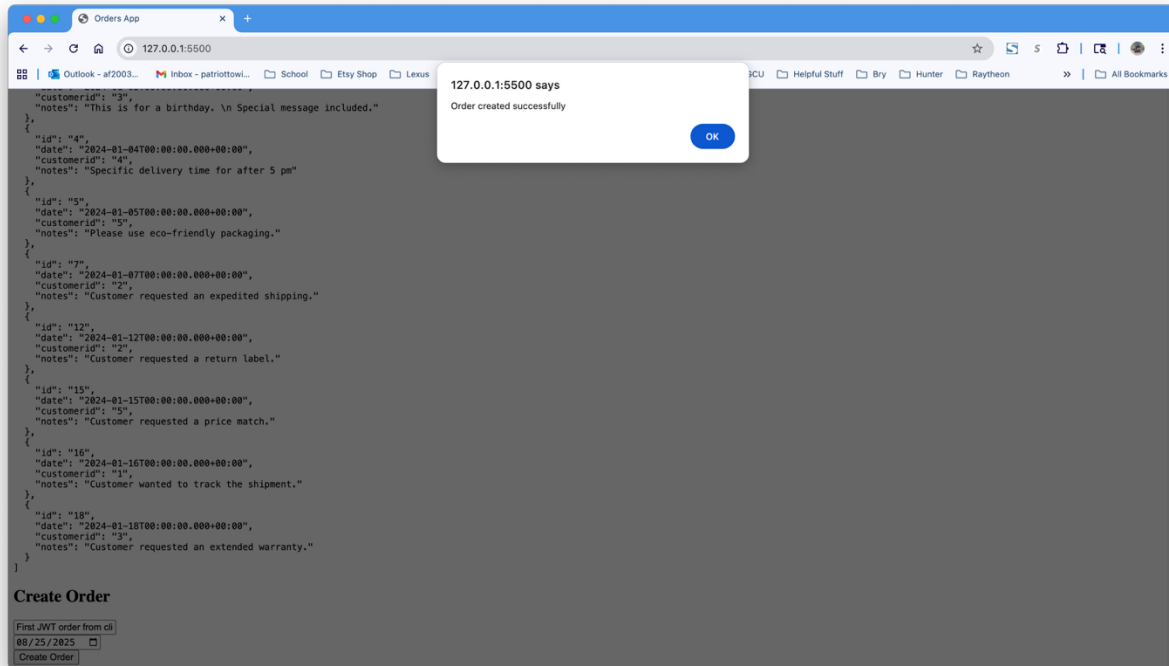


Figure 26 Created a new order from the client; request sent with `Authorization: Bearer <token>`.

Step 5 – Secure Orders API with Authenticated User Context

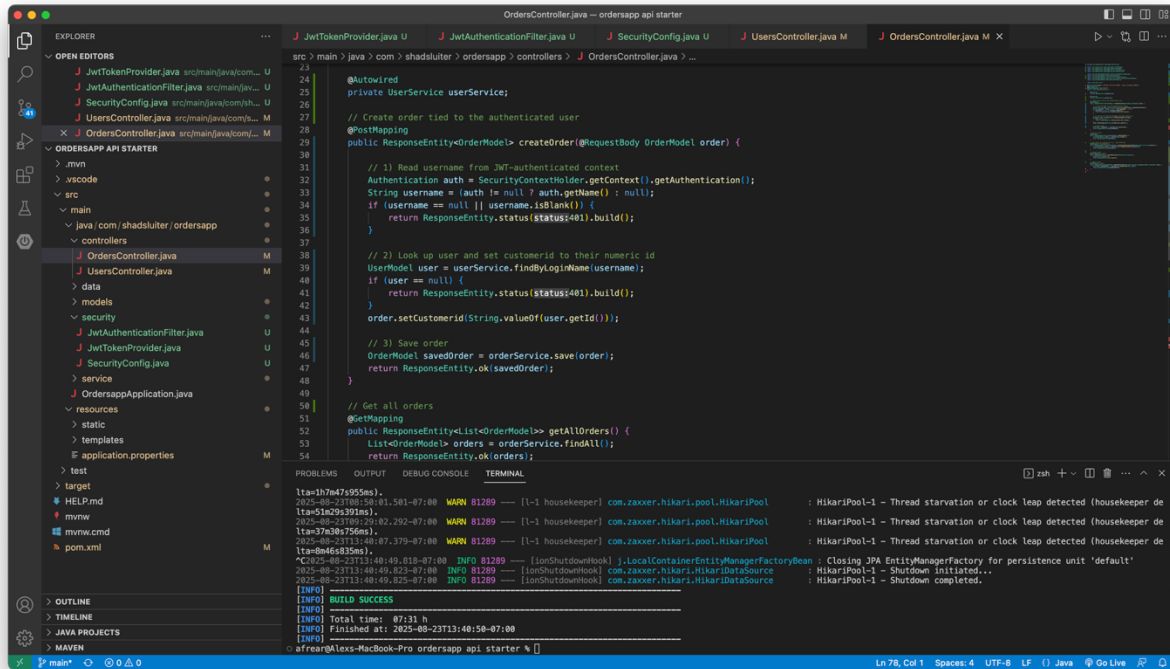


Figure 27 `OrdersController` now assigns `customerid` from the authenticated user (`username` from `JWT` $\rightarrow$ `user` lookup $\rightarrow$ `user.id`).

5.2 – Orders filtered by authenticated user

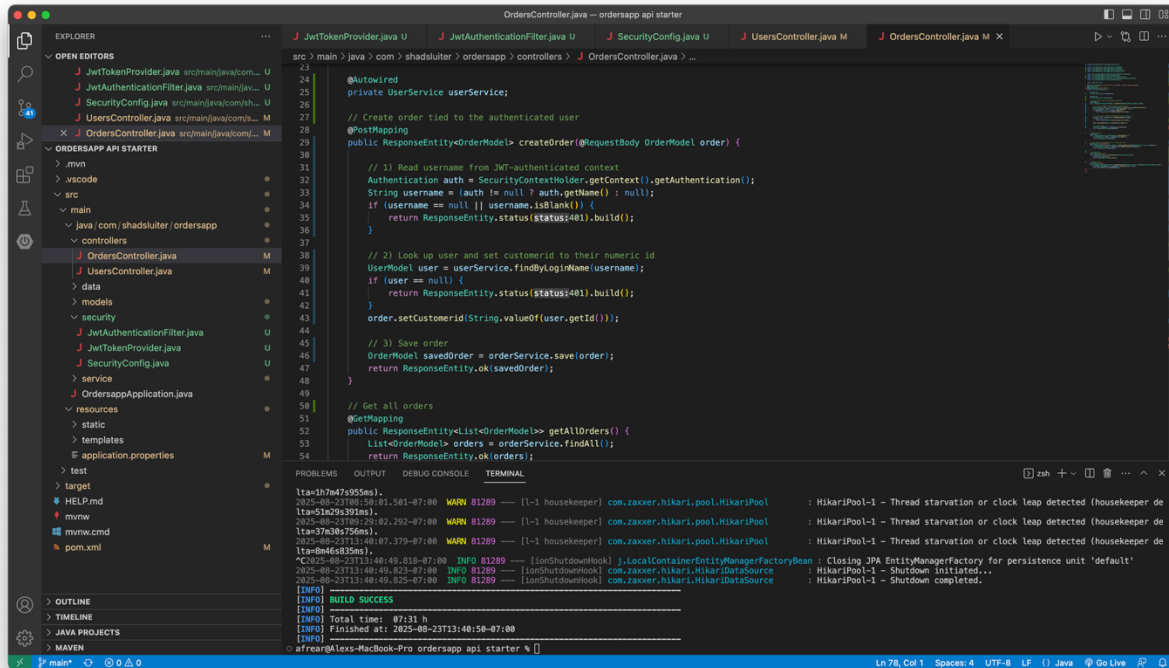


Figure 28 Orders filtered by the authenticated user's `customerId` (1003) show the newly created order tied to the logged-in account.

5.2.1 – Retrieve Orders by Customer Id (Protected)

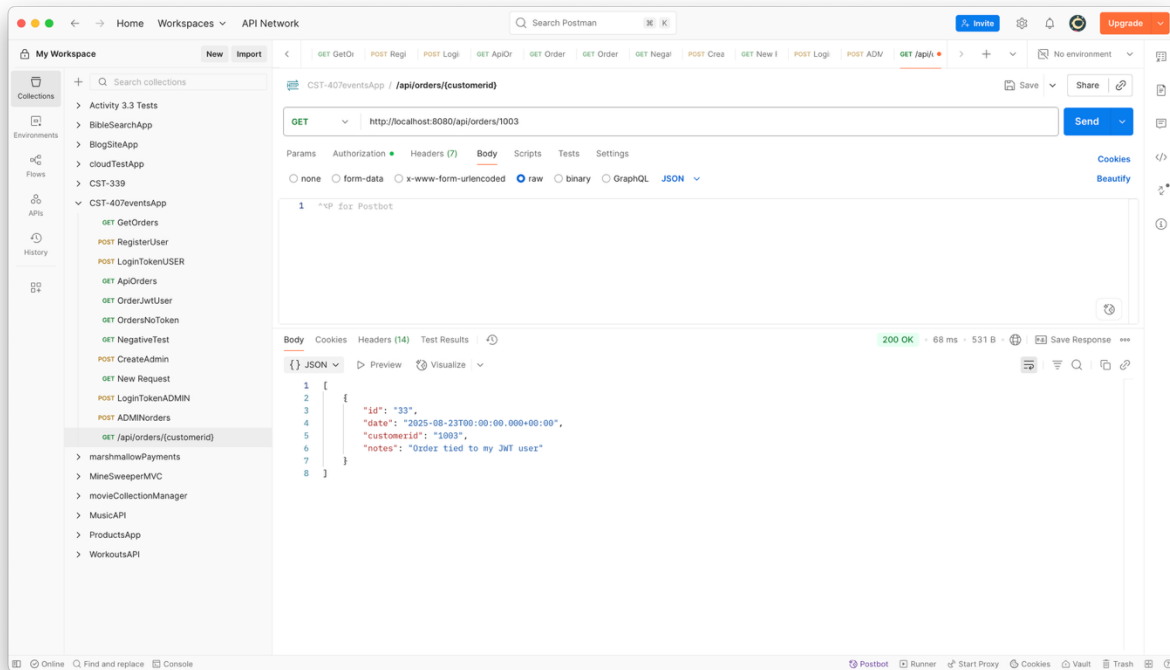


Figure 29 GET /api/orders/{customerid} returns orders for the specified customer when a valid Bearer token is provided.

5.3 — Negative test: protected endpoint without token

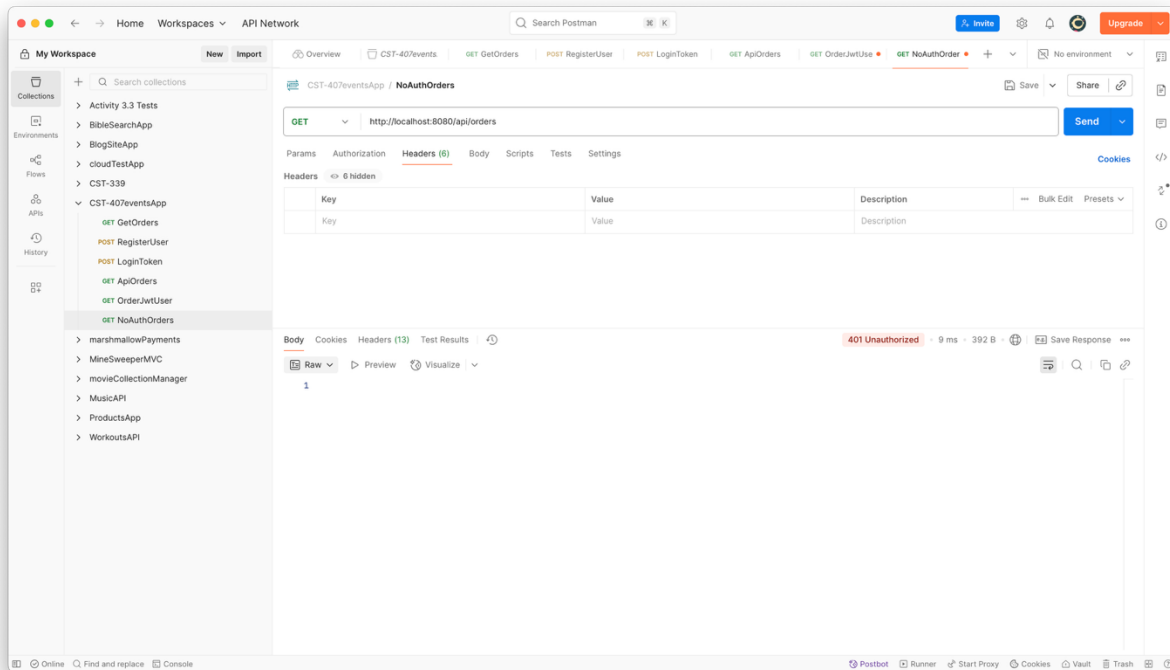


Figure 30 Protected endpoint `/api/orders` denies access without a JWT (401 Unauthorized).

Step 5.4 – Negative Test (Invalid Token)

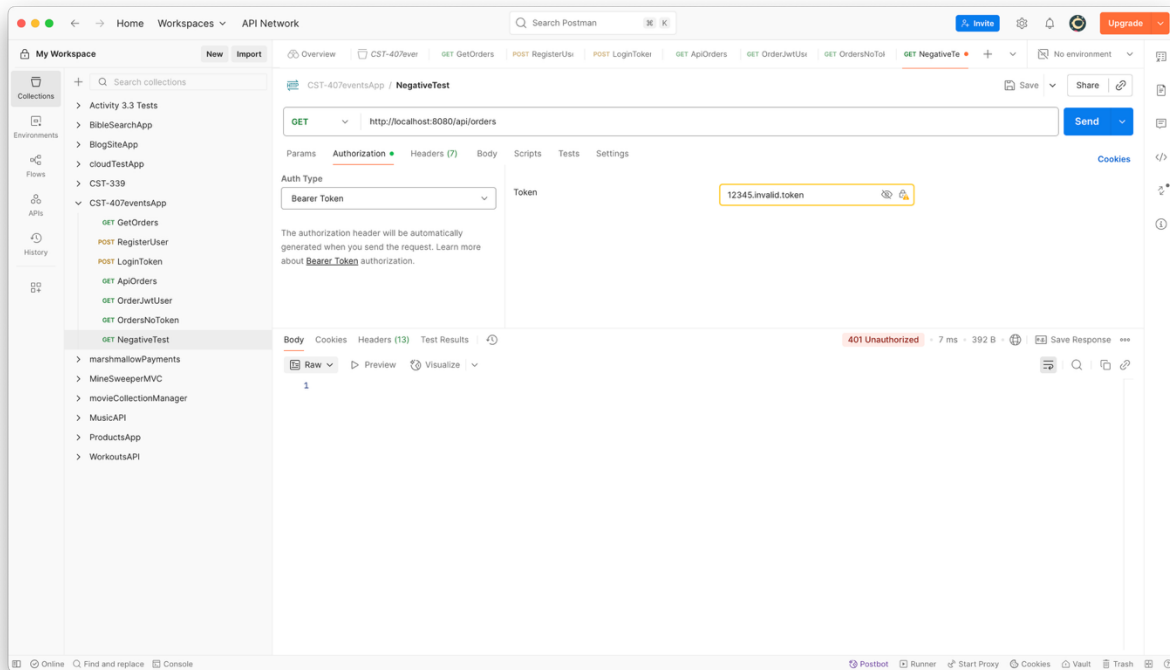


Figure 31 Protected endpoint rejects an invalid JWT with **401 Unauthorized**.

5.5 — Verify authorization from the client (not logged in)

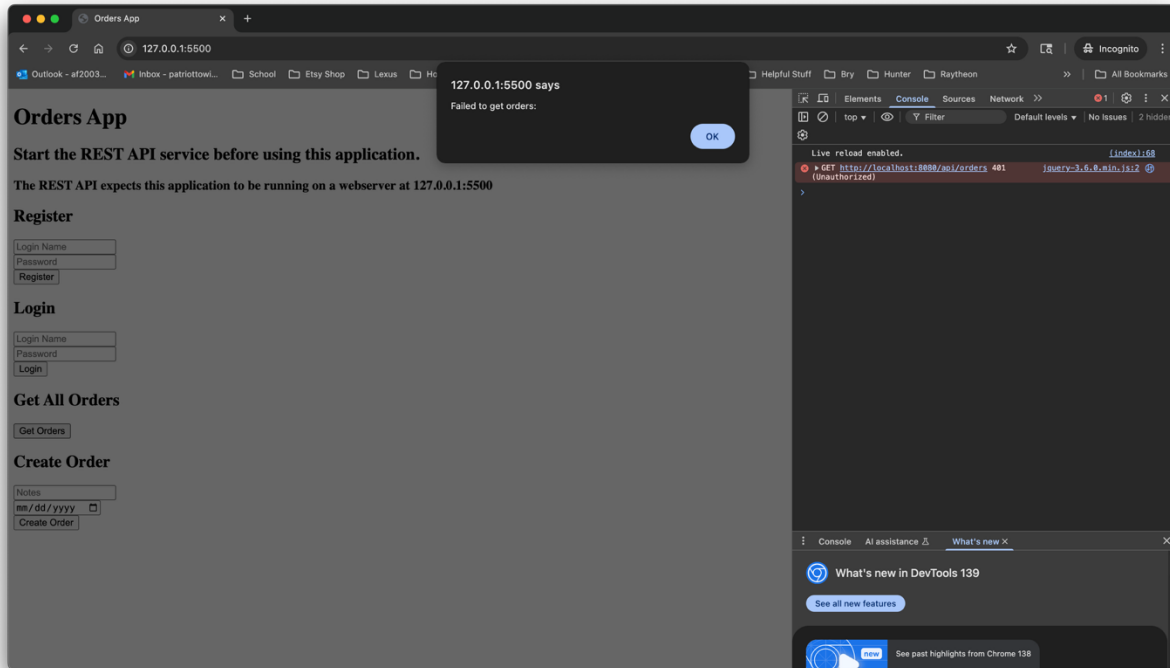


Figure 32 Client is blocked from `/api/orders` when not logged in; server returns **401 Unauthorized**.

5.6 – Verify authorization from the client (logged in)

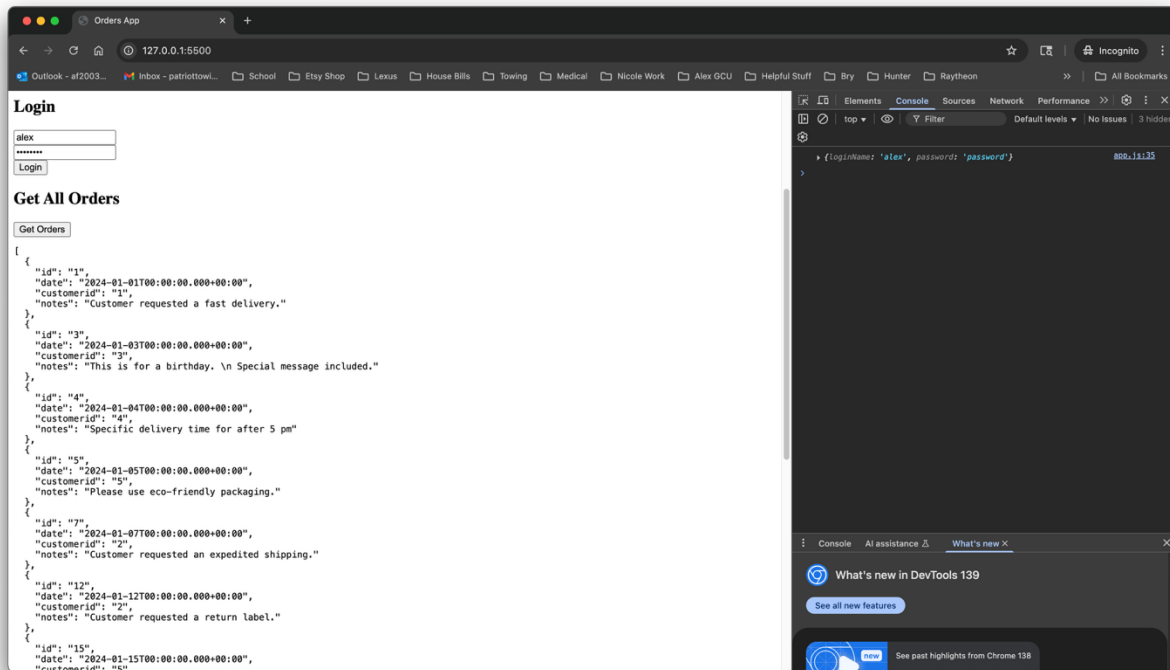


Figure 33 Client successfully retrieves orders from `/api/orders` after logging in with a valid JWT.

Step 6 — Add Role-Based Authorization

6.1 — Add a `role` field to `UserModel` (default: `USER`)

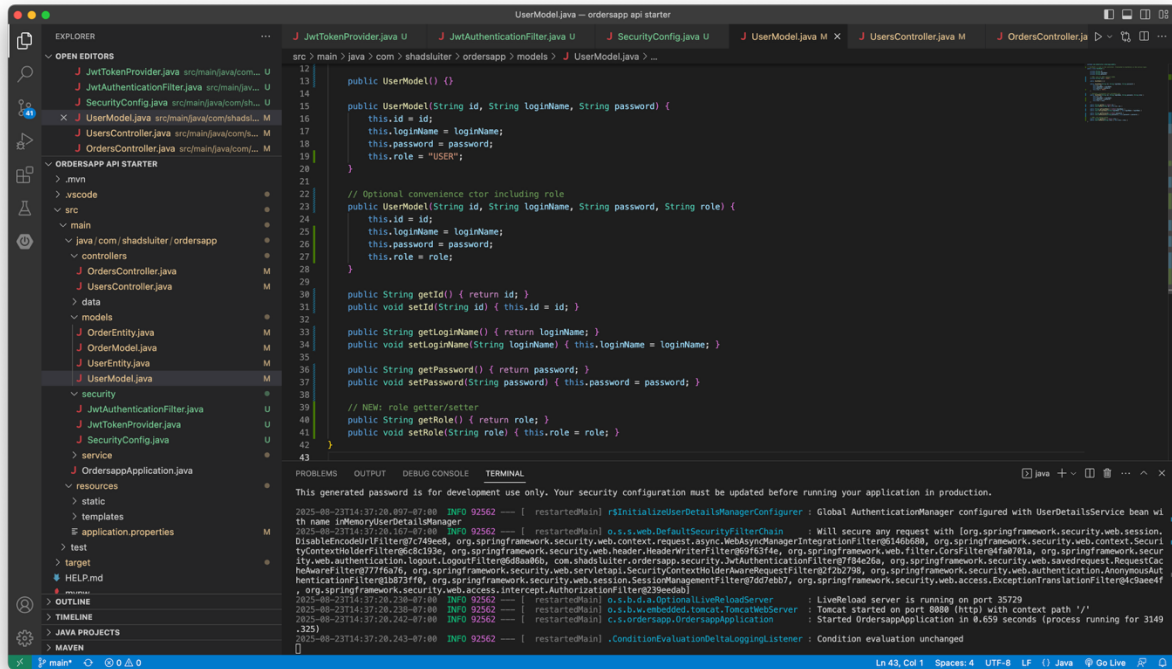


Figure 34 Added `role` to `UserModel` (default `USER`) to support role-based access control.

6.2 — Put the user's `role` in the JWT and load it into Spring Security

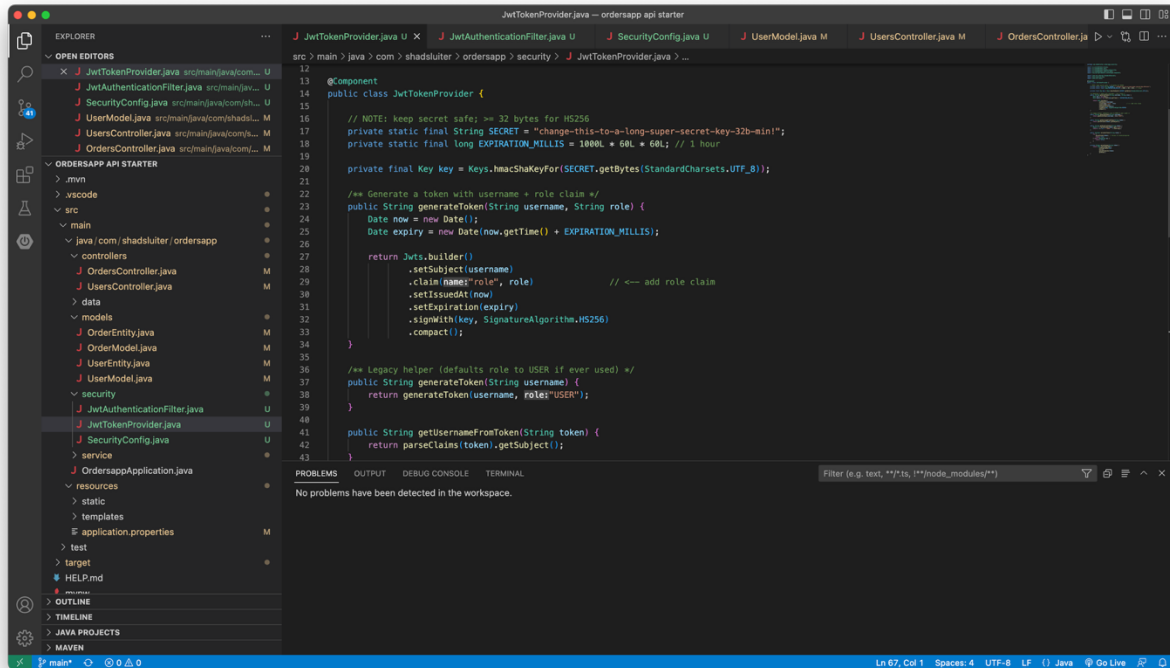


Figure 35 JWTs now include the user's `role` claim; the filter loads it as `ROLE_<role>` for authorization.

6.3 — Restrict write operations to ADMIN

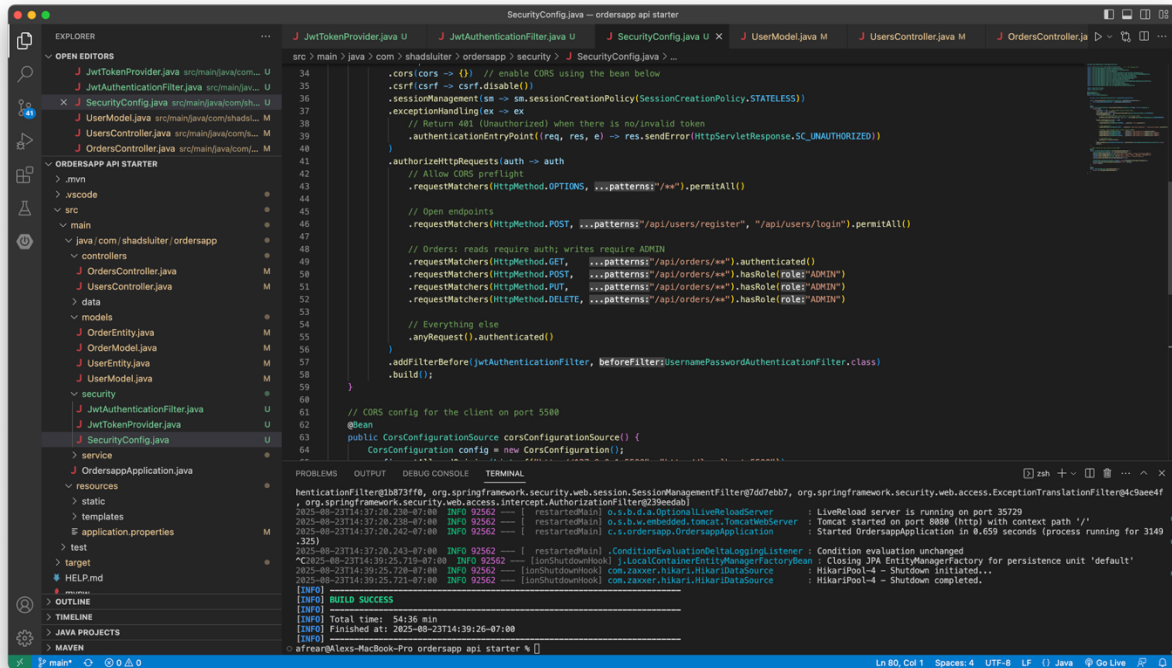


Figure 36 POST/PUT/DELETE /api/orders/** require ADMIN; reads (GET) require authentication.

6.4 Create an Admin User

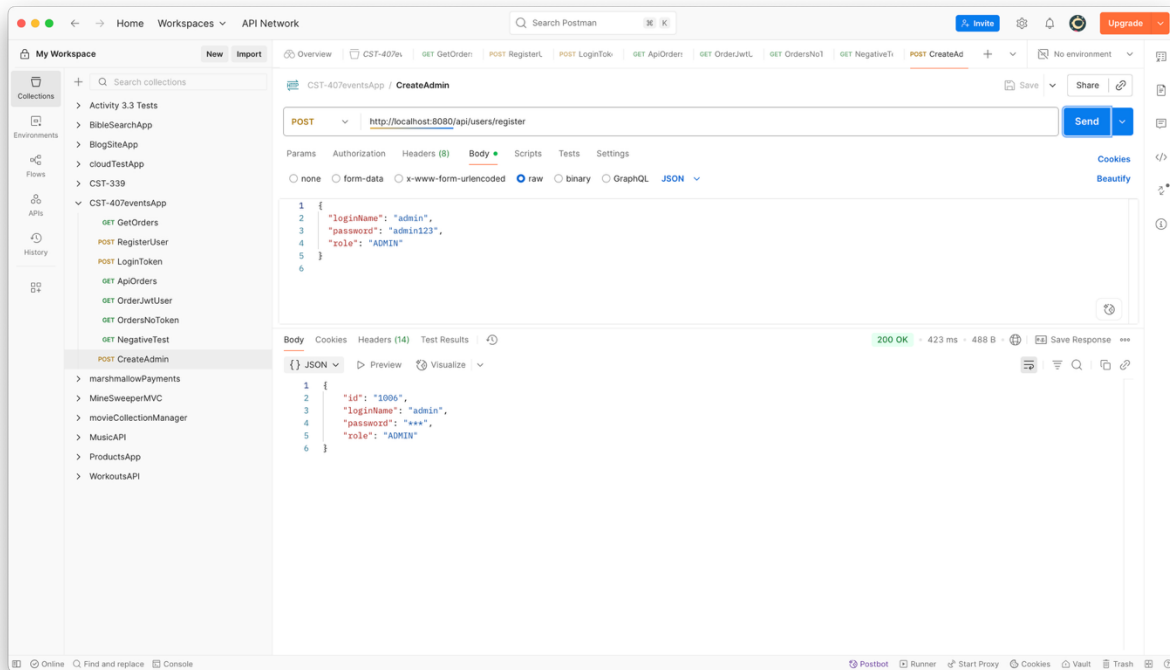


Figure 37 Postman test showing successful creation of an Admin user (200 OK response).

6.5 — USER blocked from creating order

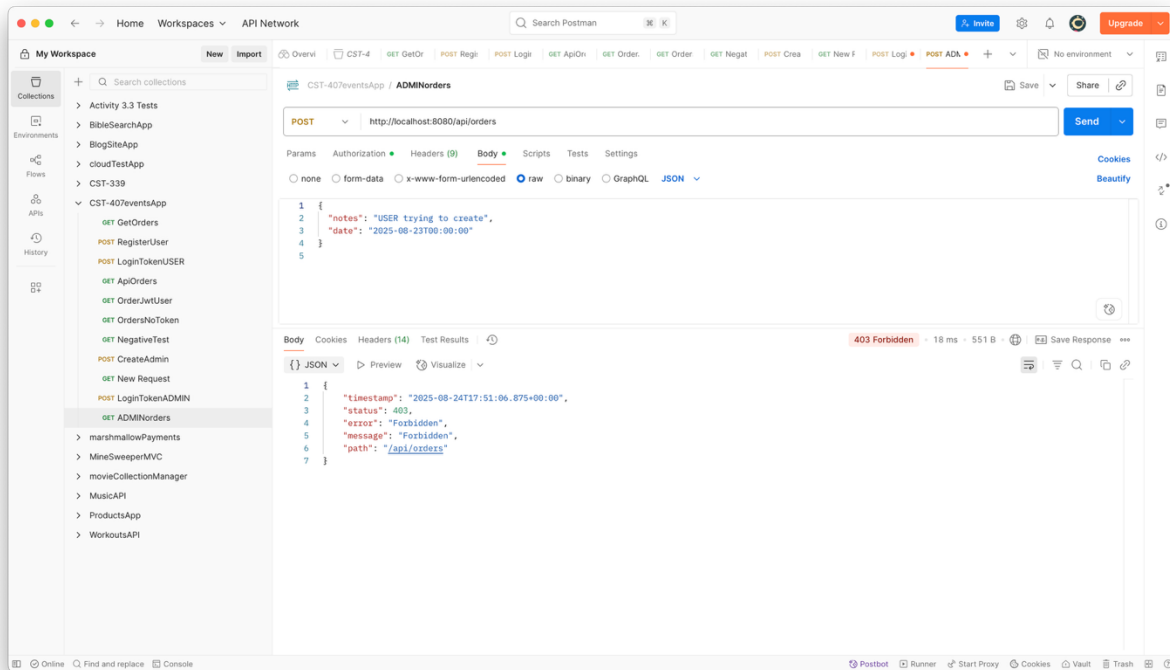


Figure 38 POST /api/orders with USER token returns **403 Forbidden**, confirming write access is restricted to ADMIN.

6.6 — Verify ADMIN *can* create an order

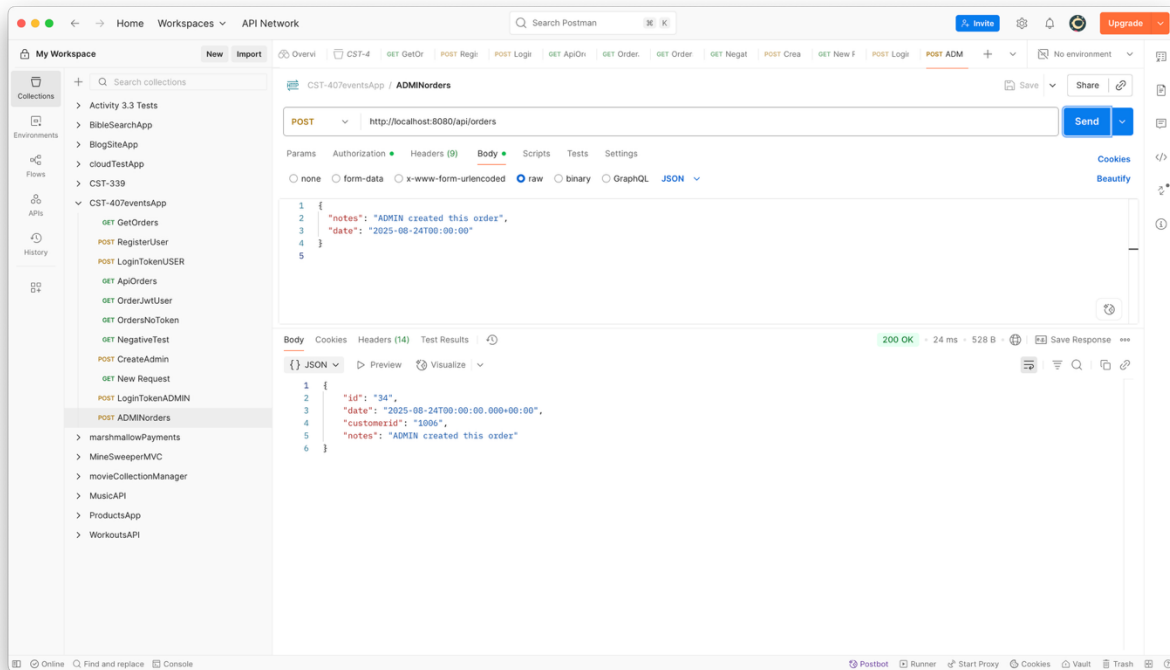


Figure 39 POST `/api/orders` with ADMIN token succeeds (200 OK), confirming ADMIN role has write permissions.

6.7 – Update Order (PUT) with Auth

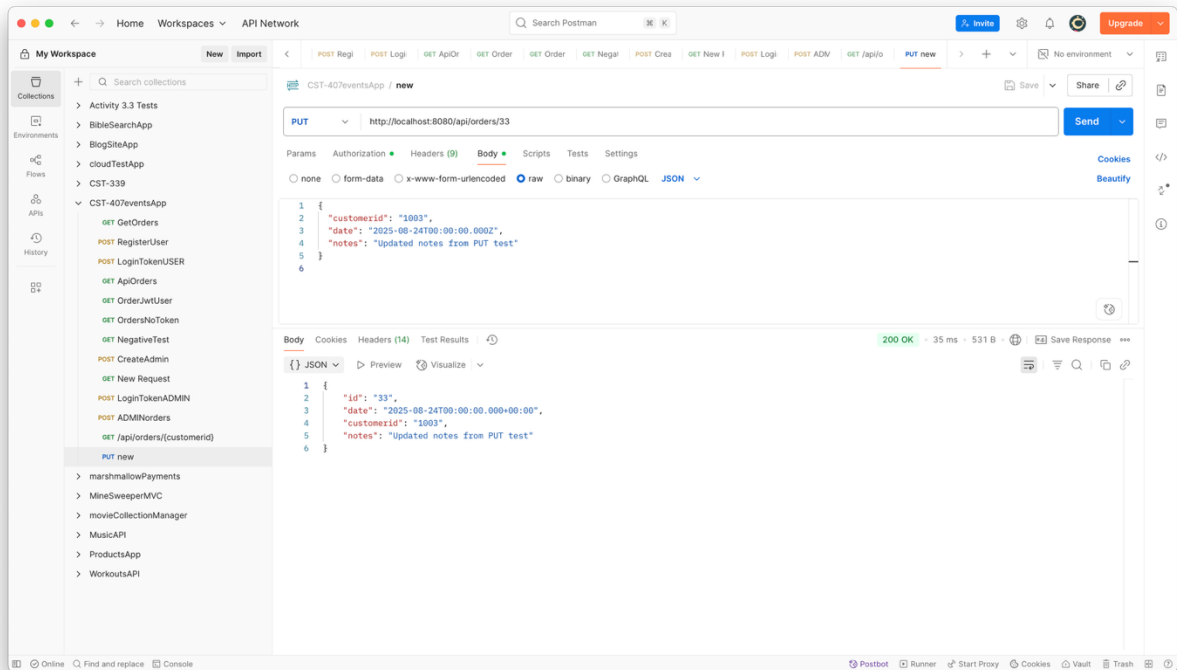


Figure 40 `/api/orders/33` updates an existing order when authorized with an ADMIN token. The response shows the `notes` field successfully updated.

6.8 – Delete Order (DELETE) with Auth

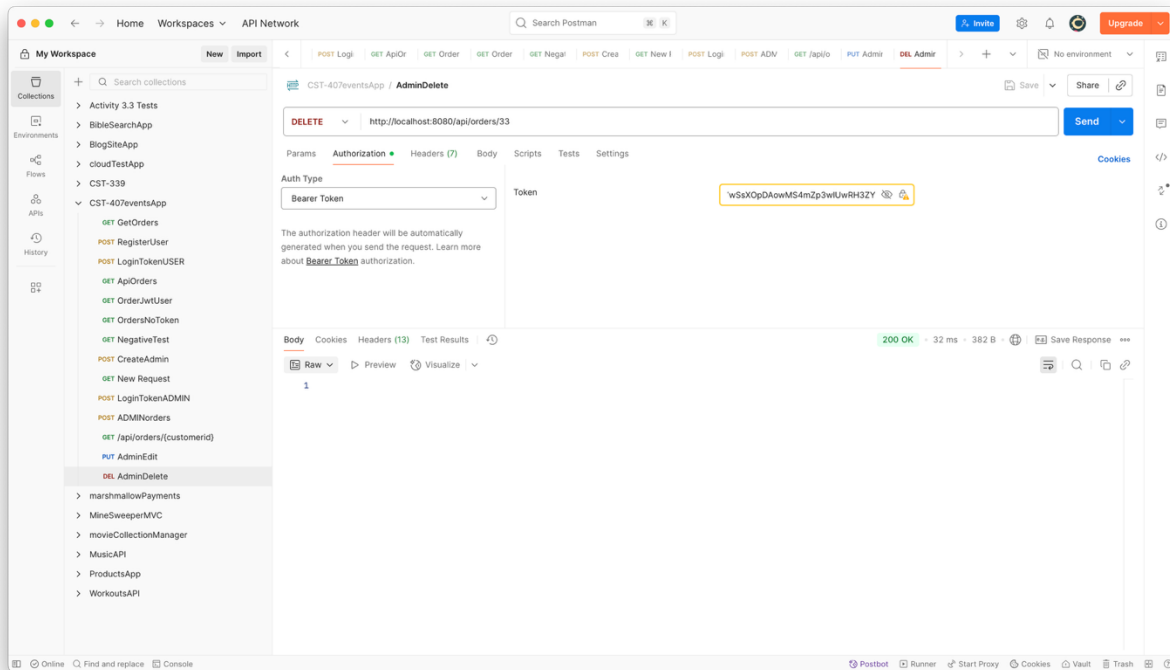


Figure 41 DELETE /api/orders/33 removes an order when authorized with an ADMIN token. The response shows 200 OK, confirming the order was deleted.

Summary of Key Concepts

In this activity, I learned how Spring Security enforces **role-based access control (RBAC)** using JWT. Each user's role (e.g., `USER` or `ADMIN`) is embedded in the JWT as a claim and extracted by the authentication filter. The security configuration uses methods like `.hasRole("ADMIN")` to restrict access so only administrators can create, update, or delete orders. Regular users are limited to reading their own orders, and any attempt to go beyond those permissions correctly results in **403 Forbidden**. This approach ensures that sensitive operations are only available to authorized accounts and demonstrates how JWT, role claims, and Spring Security work together to provide fine-grained access control.